

操作系统面试题，34道操作系统八股文（1万字51张手绘图），面渣逆袭必看👉

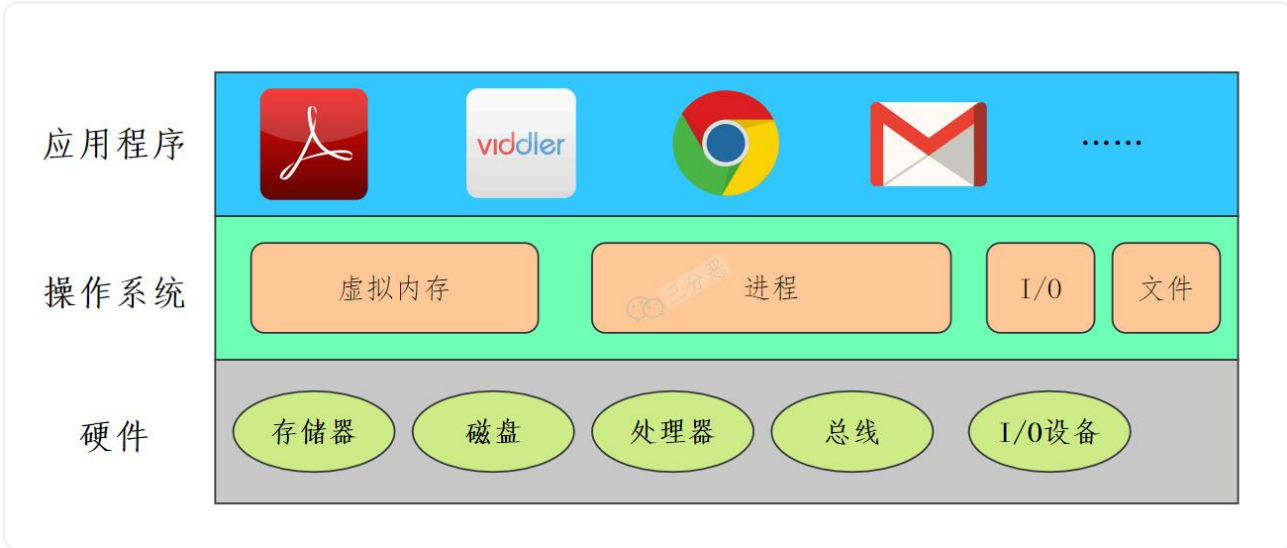
👤 三分恶 📅 2022年6月14日 📖 面渣逆袭 🔍 面渣逆袭 📄 约 12661 字 ⌚ 大约 42 分钟

1 万字 51 张手绘图，详解 34 道操作系统面试高频题（让天下没有难背的八股），面渣背会这些 OS 八股文，这次吊打面试官，我觉得稳了（手动 dog）。整理：沉默王二，戳[转载链接 \(https://mp.weixin.qq.com/s/CYsn0M5ddDuG--mALmhsuw\)](https://mp.weixin.qq.com/s/CYsn0M5ddDuG--mALmhsuw)，作者：三分恶，戳[原文链接 \(https://mp.weixin.qq.com/s/KMGyn-FLkvzsMH06LV4OfQ\)](https://mp.weixin.qq.com/s/KMGyn-FLkvzsMH06LV4OfQ)。

引论

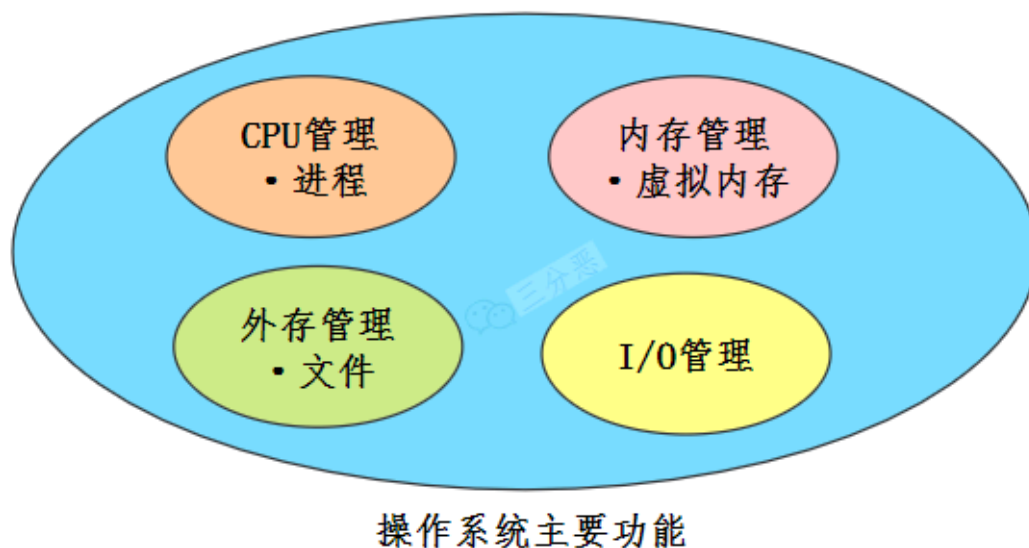
01、什么是操作系统？

操作系统（Operating System, OS）是计算机系统中管理硬件和软件资源的中间层系统，屏蔽了硬件的复杂性，并且为用户提供了便捷的交互方式，比如说 Windows、Linux、MacOS 等。



三分恶面渣逆袭：操作系统是什么

02、操作系统主要有哪些功能？



三分恶面渣逆袭：操作系统主要功能

- ①、负责创建和终止进程。进程是正在运行的程序实例，每个进程都有自己的地址空间和资源。
- ②、负责为进程分配资源，比如说内存，并在进程终止时回收内存。
- ③、提供创建、删除、读写文件的功能，并组织文件的存储结构，比如说目录。
- ④、通过设备驱动程序控制和管理计算机的硬件设备，如键盘、鼠标、打印机等。

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的比亚迪面经同学 1 面试原题：操作系统的主要功能，以及上下文切换

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer) (<https://github.com/itwanger/toBeBetterJavaer>)》第一版 PDF 终于来了！包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等，共计 32 万余字，500+张手绘图，可以说是通俗易懂、风趣幽默.....详情戳：[太赞了，GitHub 上标星 10000+ 的 Java 教程](https://javabetter.cn/overview/) (<https://javabetter.cn/overview/>)

微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二，回复 **222** 即可免费领取。



操作系统结构

03、什么是内核？

可以这么说，内核是一个计算机程序，它是操作系统的核心，提供了操作系统最核心的能力，可以控制操作系统中所有的内容。

04、什么是用户态和内核态？

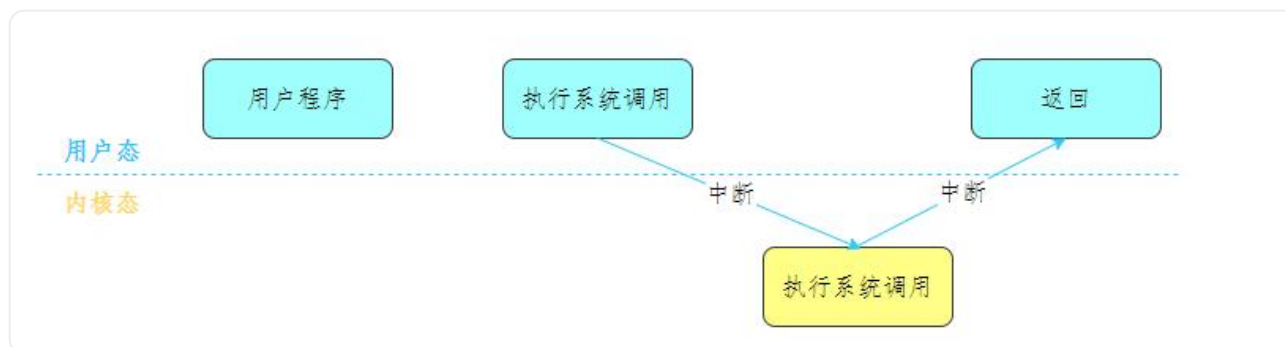
内核具有很高的权限，可以控制 cpu、内存、硬盘等硬件，出于权限控制的考虑，因此大多数操作系统，把内存分成了两个区域：

- 内核空间，这个内存空间只有内核程序可以访问；
- 用户空间，这个内存空间专门给应用程序使用，权限比较小；

用户空间的代码只能访问一个局部的内存空间，而内核空间的代码可以访问所有内存空间。因此，当程序使用用户空间时，我们常说该程序在**用户态**执行，而当程序使用内核空间时，程序则在**内核态**执行。

05、用户态和内核态是如何切换的？

应用程序如果需要进入内核空间，就需要通过系统调用，来进入内核态：



用户态&内核态切换

内核程序执行在内核态，用户程序执行在用户态。当应用程序使用系统调用时，会产生一个中断。发生中断后，CPU 会中断当前在执行的程序，转而跳转到中断处理程序，也就是开始执行内核程序。内核处理完后，主动触发中断，把 CPU 执行权限交回给用户程序，回到用户态继续工作。

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer) (<https://github.com/itwanger/toBeBetterJavaer>)》第一版 PDF 终于来了！包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等，共计 32 万余字，500+张手绘图，可以说是通俗易懂、风趣幽默.....详情戳：[太赞了，GitHub 上标星 10000+ 的 Java 教程](https://javabetter.cn/overview/) (<https://javabetter.cn/overview/>)

微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二，回复 **222** 即可免费领取。

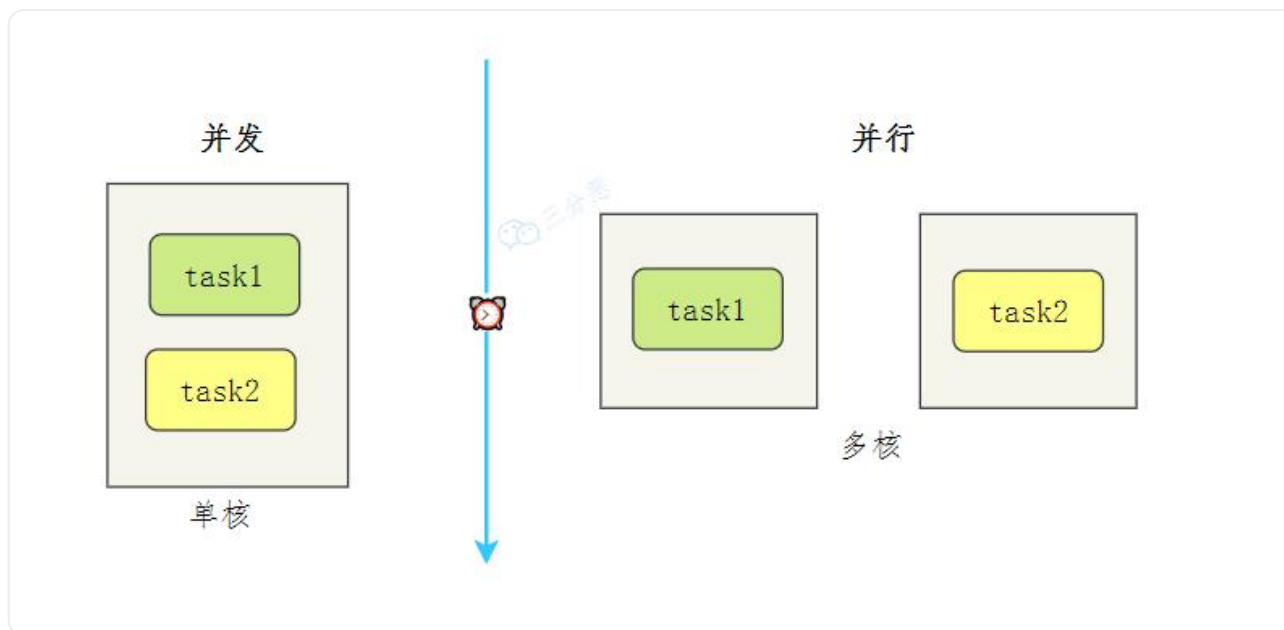


进程和线程

06、并行和并发有什么区别？

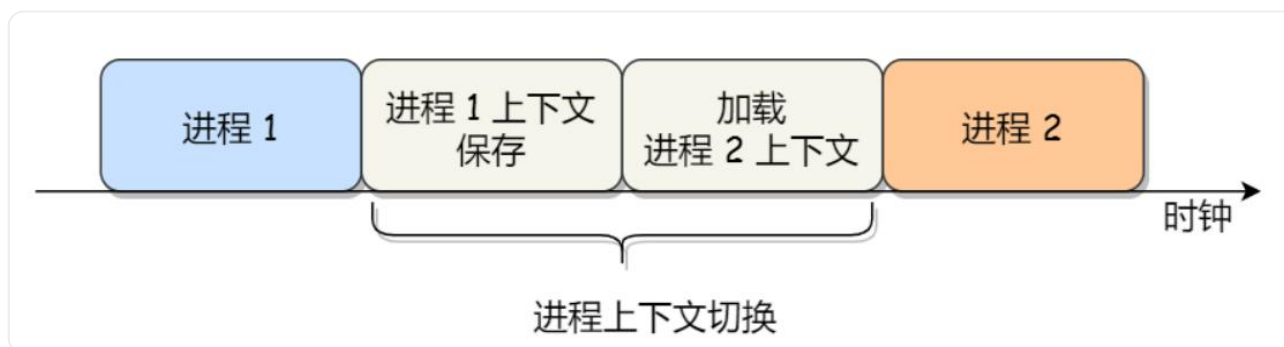
并发就是在一段时间内，多个任务都会被处理；但在某一时刻，只有一个任务在执行。单核处理器做到的并发，其实是利用时间片的轮转，例如有两个进程 A 和 B，A 运行一个时间片之后，切换到 B，B 运行一个时间片之后又切换到 A。因为切换速度足够快，所以宏观上表现为在一段时间内能同时运行多个程序。

并行就是在同一时刻，有多个任务在执行。这个需要多核处理器才能完成，在微观上就能同时执行多条指令，不同的程序被放到不同的处理器上运行，这个是物理上的多个进程同时进行。



并发和并行

07、什么是进程上下文切换？



三分恶面渣逆袭：进程上下文切换

上下文切换是操作系统在多任务处理环境中，将 CPU 从一个进程切换到另一个进程的过程。通过让多个进程共享 CPU 资源，使系统能够并发执行多个任务。

进程上下文切换通常包含以下几个步骤：

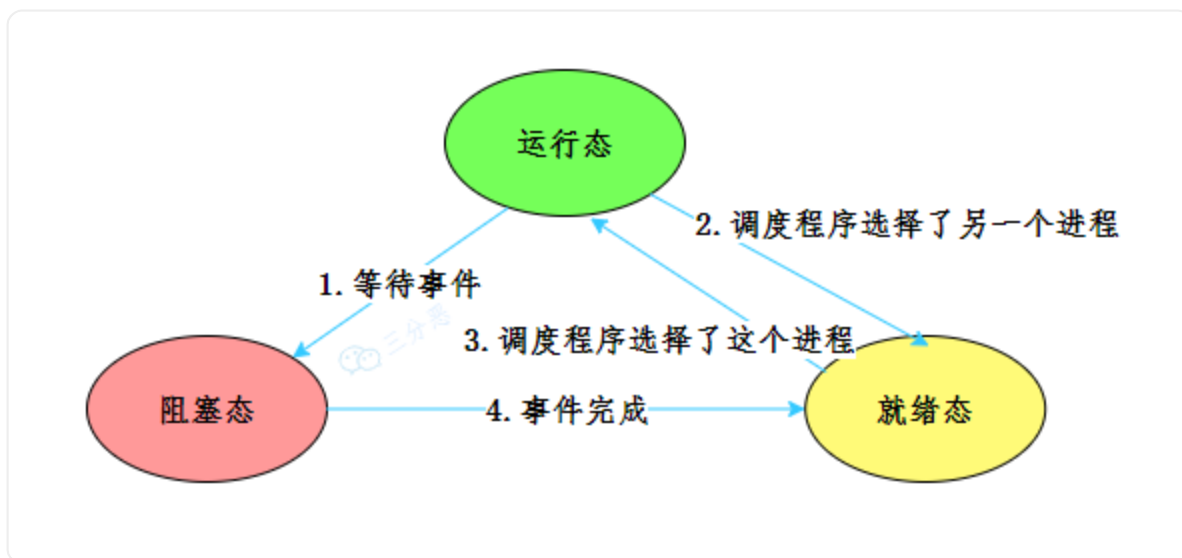
- 保存当前进程的上下文：操作系统保存当前进程的 CPU 寄存器，程序状态等关键信息。
- 选择下一个进程：调度程序选择下一个要执行的进程。
- 恢复上一个进程的上下文。
- 切换到下一个进程。

08、进程有哪些状态？

当一个进程开始运行时，它可能会经历下面这几种状态：

上图中各个状态的意义：

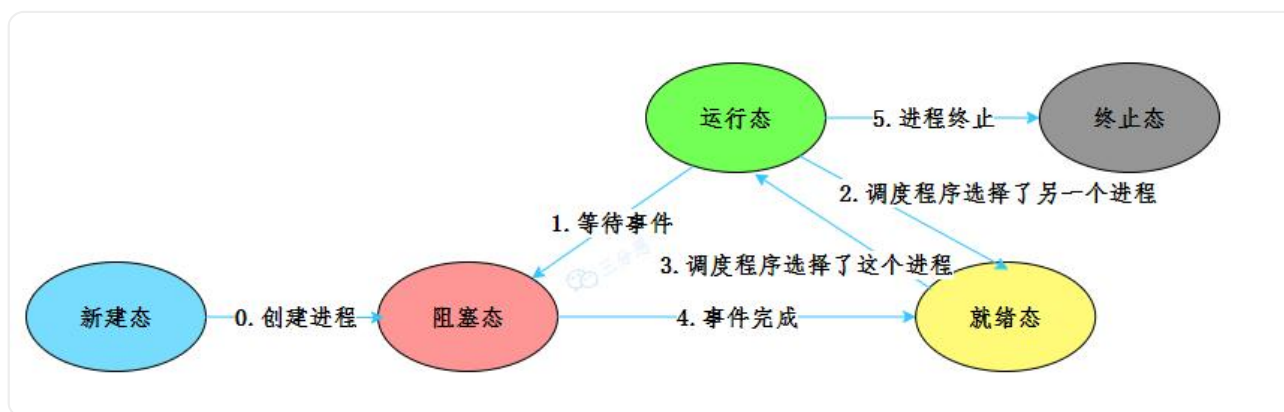
- 运行状态 (*Runing*)：该时刻进程占用 CPU；
- 就绪状态 (*Ready*)：可运行，由于其他进程处于运行状态而暂时停止运行；
- 阻塞状态 (*Blocked*)：该进程正在等待某一事件发生（如等待输入/输出操作的完成）而暂时停止运行，这时，即使给它 CPU 控制权，它也无法运行；



进程3种状态

当然，进程还有另外两个基本状态：

- 创建状态 (*new*)：进程正在被创建时的状态；
- 结束状态 (*Exit*)：进程正在从系统中消失时的状态；



进程5种状态

09、什么是僵尸进程？

僵尸进程是已完成且处于终止状态，但在进程表中却仍然存在的进程。

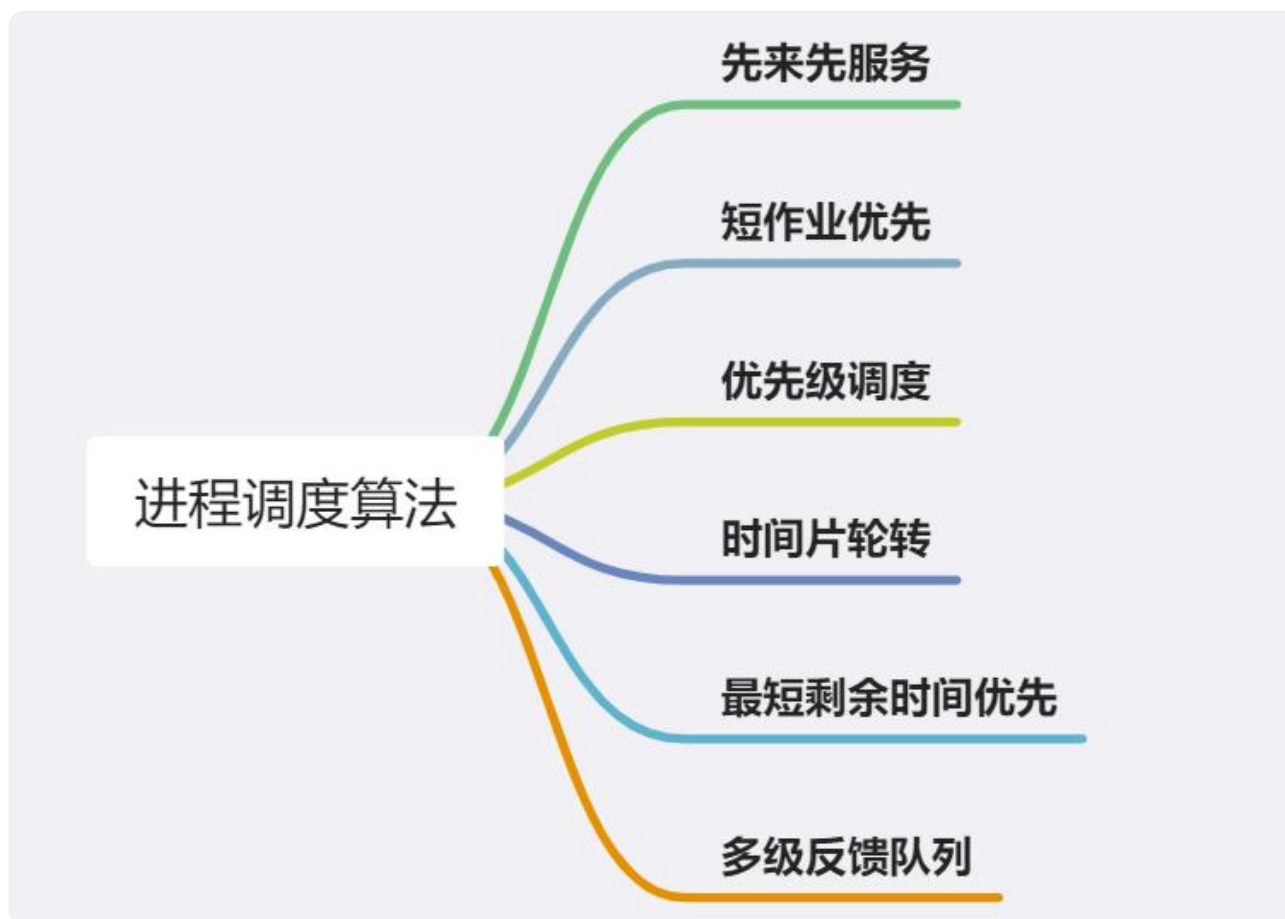
僵尸进程一般发生在有父子关系的进程中，一个子进程的进程描述符在子进程退出时不会释放，只有当父进程通过 `wait()` 或 `waitpid()` 获取了子进程信息后才会释放。如果子进程退出，而父进程并没有调用 `wait()` 或 `waitpid()`，那么子进程的进程描述符仍然保存在系统中。

10、什么是孤儿进程？

一个父进程退出，而它的一个或多个子进程还在运行，那么这些子进程将成为孤儿进程。孤儿进程将被 `init` 进程（进程 ID 为 1 的进程）所收养，并由 `init` 进程对它们完成状态收集工作。因为孤儿进程会被 `init` 进程收养，所以孤儿进程不会对系统造成危害。

11、进程有哪些调度算法？

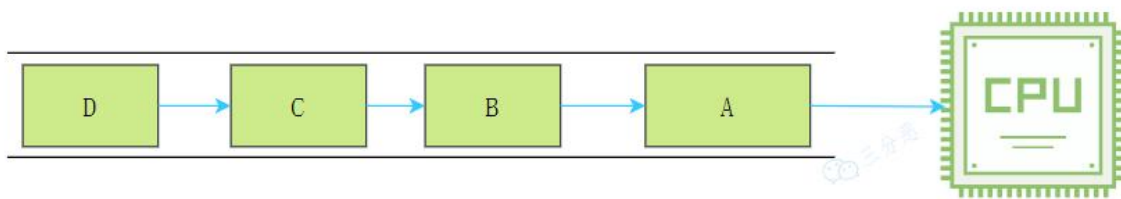
进程调度是操作系统中的核心功能之一，它负责决定哪些进程在何时使用 CPU。这一决定基于系统中的进程调度算法。



DIDA-IJ-进程调度算法

①、先来先服务

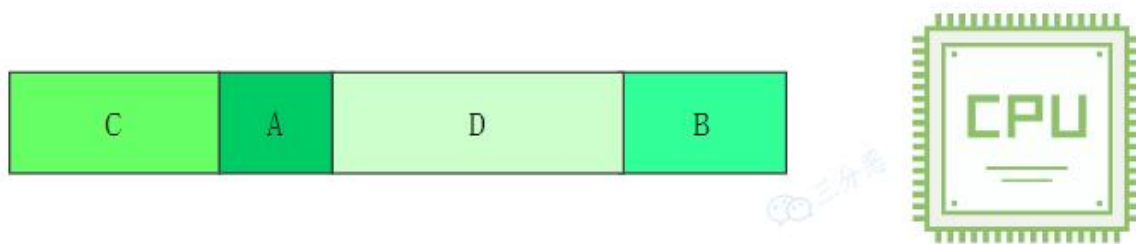
这是最简单的调度算法，也称为先进先出（FIFO）。进程按照请求 CPU 的顺序进行调度。这种方式易于实现，但可能会导致较短的进程等待较长进程执行完成，从而产生“饥饿”现象。



三分恶面渣逆袭：先来先服务

②、短作业优先

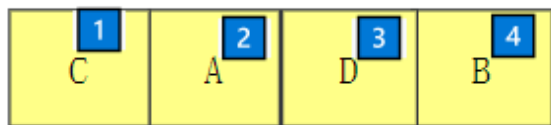
选择预计运行时间最短的进程优先执行。这种方式可以减少平均等待时间和响应时间，但缺点是很难准确预知进程的执行时间，并且可能因为短作业一直在执行，导致长作业持续被推迟执行。



三分恶面渣逆袭：短作业优先

③、优先级调度

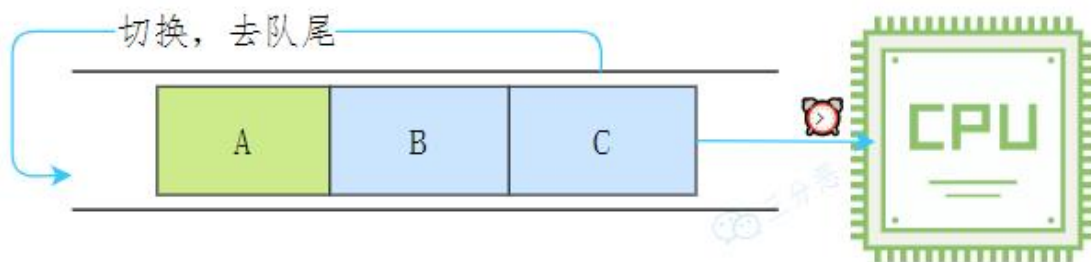
在这种调度方式中，每个进程都被分配一个优先级。CPU 首先分配给优先级最高的进程。优先级调度可以是非抢占式的或抢占式的。在非抢占式优先级调度中，进程一旦开始执行将一直运行直到完成；在抢占式优先级调度中，更高优先级的进程可以中断正在执行的低优先级进程。



三分恶面渣逆袭：优先级调度

④、时间片轮转

时间片轮转调度为每个进程分配一个固定的时间段，称为时间片，进程可以在这个时间片内运行。如果进程在时间片结束时还没有完成，它将被放回队列的末尾。时间片轮转是公平的调度方式，可以保证所有进程得到公平的 CPU 时间，适用于共享系统。



三分恶面渣逆袭：时间片轮转

⑤、最短剩余时间优先

这是短作业优先的一种改进形式，它是抢占式的。即如果一个新进程的预计执行时间比当前运行进程的剩余时间短，调度器将暂停当前的进程，并切换到新进程。这种方法也可以最小化平均等待时间，但同样面临预测执行时间的困难。

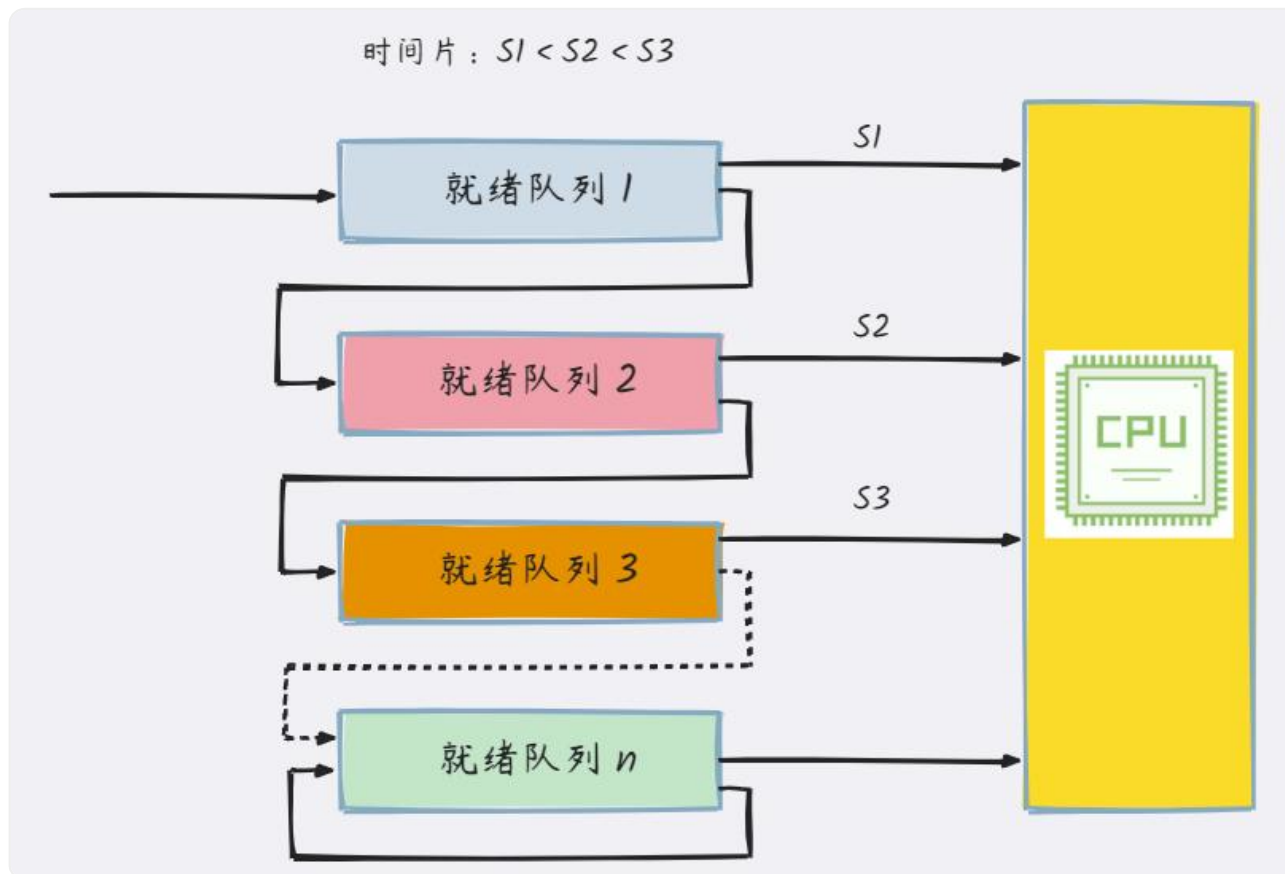
⑥ 多级反馈队列

一个进程需要执行100 哥时间片，如果采用时间片轮转调度算法，那么需要交互 100 次。

多级队列就是为这种需要连续执行多个时间片的进程考虑，它设置了多个队列，每个队列的时间片大小不同，比如 2,4,6,8……。进程在第一个队列没执行完，就会被移到下一个队列。

这种方式下，之前的进程只需要交换 7 次就可以了。每个队列优先权不一样，最上面的队列优先权最高。因此只有上一个队列没有进程在排队，才能调度当前队列上的进程。

可以将这种调度算法看成是时间片轮转调度算法与优先级调度算法的结合。



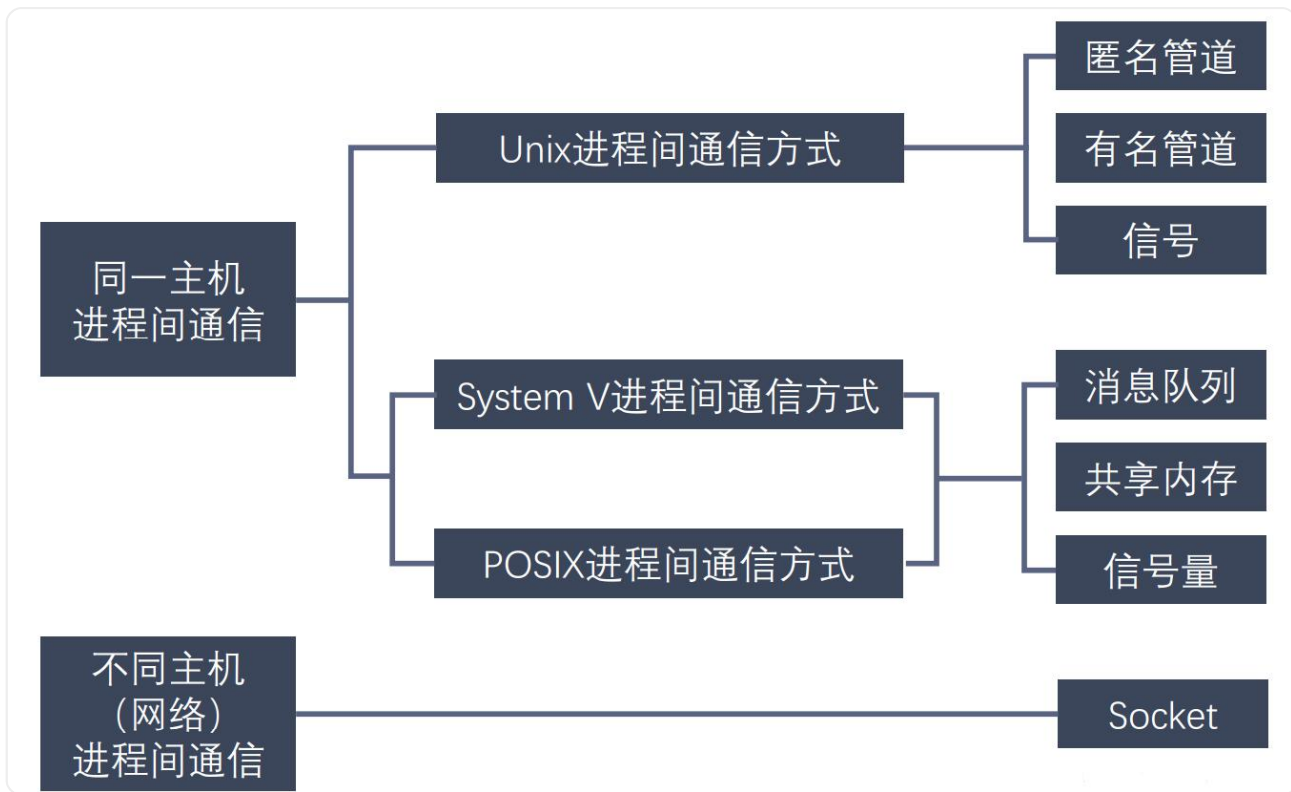
DIDA-II-多级反馈队列

1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的华为面经同学 9 Java 通用软件开发一面面试原题：进程的调度方式

12、进程间通信有哪些方式？

推荐阅读：[编程十万问：进程间通信的方式有哪些？](https://www.nowcoder.com/discuss/536674147976470528)
(<https://www.nowcoder.com/discuss/536674147976470528>)

进程间通信（IPC，Inter-Process Communication）的方式有管道、信号、消息队列、共享内存、信号量和套接字。



编程十万问：进程间通信

简单说说管道：

管道可以理解成不同进程之间的传话筒，一方发声，一方接收，声音的介质可以是空气或者电缆。

进程间的管道就是内核中的一串缓存，从管道的一端写入数据，另一端读取。数据只能单向流动，遵循先进先出（FIFO）的原则。

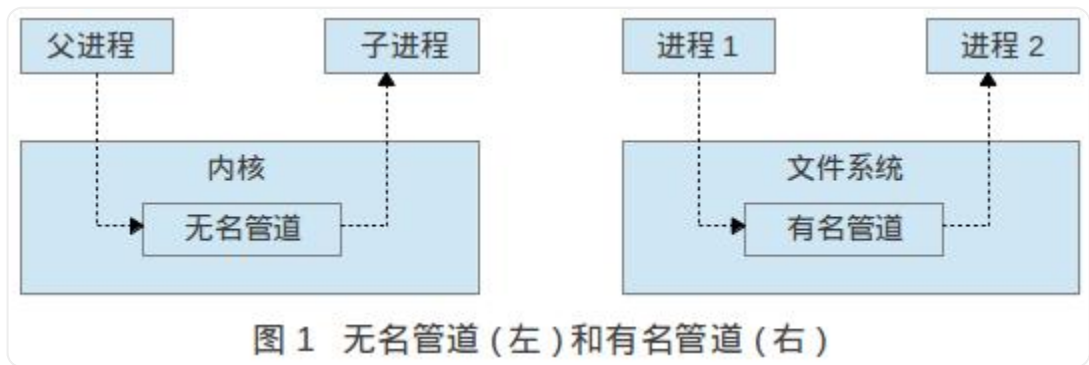


图 1 无名管道（左）和有名管道（右）

编程十万问：管道

①、**匿名管道**：允许具有亲缘关系的进程（如父子进程）进行通信。

"奉先我儿" 就可以理解成一种命名管道，董卓叫亲儿子，直接叫“我儿”就可以了



三分恶面渣逆袭：“奉先我儿”

使用 C 语言在 Unix/Linux 环境下通过匿名管道实现两个进程（通常是父子进程）之间通信的示例：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    int pipefd[2];
    pid_t cpid;
    char buf;

    // 创建管道
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // 创建子进程
    cpid = fork();
    if (cpid == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
```

C

```
if (cpid == 0) {    /* 子进程 */
    close(pipefd[1]); // 关闭写端

    // 从管道读取数据
    while (read(pipefd[0], &buf, 1) > 0)
        write(STDOUT_FILENO, &buf, 1);

    write(STDOUT_FILENO, "\n", 1);
    close(pipefd[0]);
    exit(EXIT_SUCCESS);
} else {            /* 父进程 */
    close(pipefd[0]); // 关闭读端

    // 向管道写入数据
    write(pipefd[1], "Hello, Child!", 13);
    close(pipefd[1]); // 关闭写端，触发EOF
    wait(NULL);        // 等待子进程退出
    exit(EXIT_SUCCESS);
}
}
```

②、**命名管道**：允许无亲缘关系的进程通信，通过在文件系统中创建一个特殊类型的文件来实现。

缺点：管道的效率低，不适合进程间频繁地交换数据。

简单说说信号：

信号可以理解成以前的 BB 机，用于通知接收进程某件事情发生了，是一种较为简单的通信方式，主要用于处理异步事件。

比如 `kill -9 1050` 就表示给 PID 为 1050 的进程发送 SIGKILL 信号。

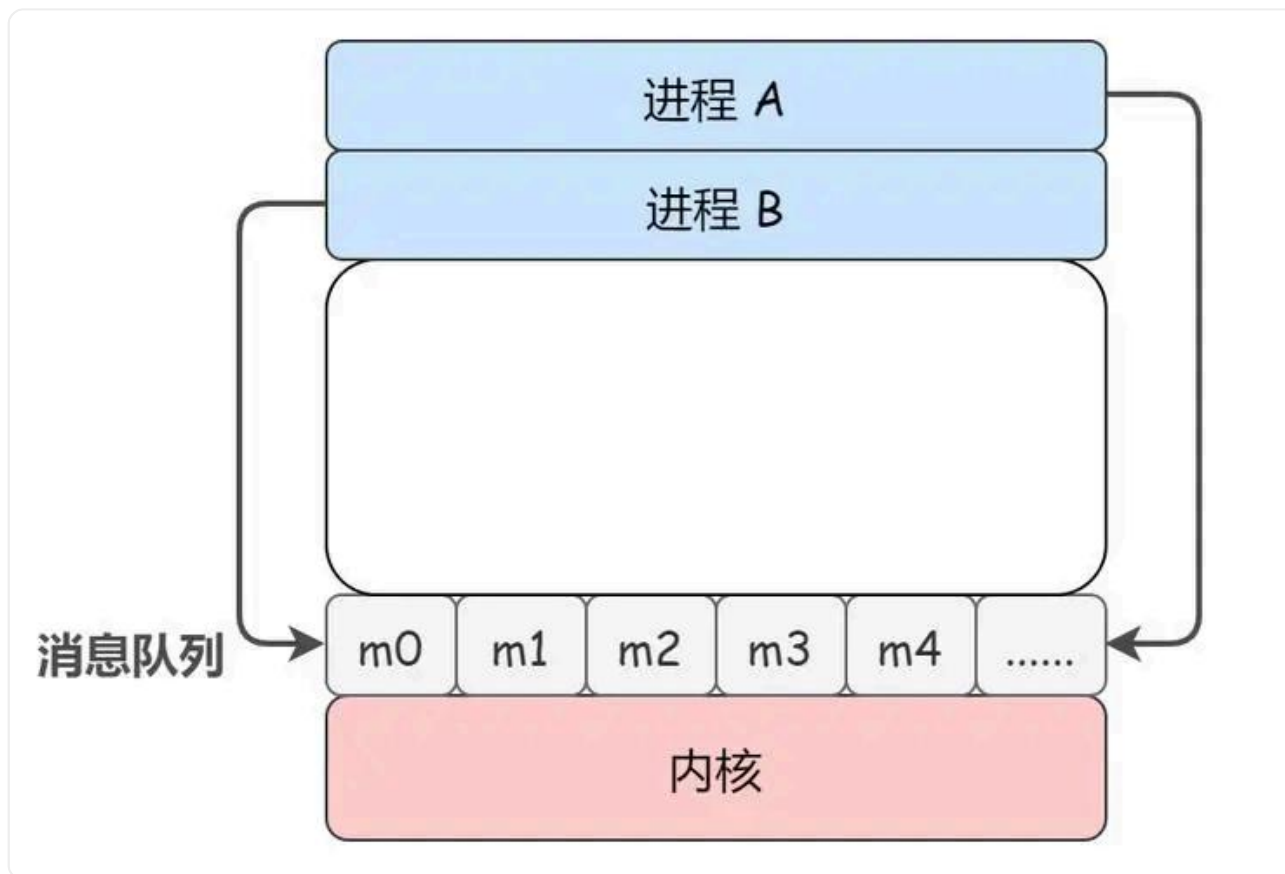
这里顺带普及一下 Linux 中常用的信号：

- **SIGHUP**：当我们退出终端（Terminal）时，由该终端启动的所有进程都会接收到这个信号，默认动作为终止进程。
- **SIGINT**：程序终止（interrupt）信号。按 `Ctrl+C` 时发出，大家应该在操作终端时有过这种操作。
- **SIGQUIT**：和 SIGINT 类似，按 `Ctrl+\` 键将发出该信号。它会产生核心转储文件，将内存映像和程序运行时的状态记录下来。
- **SIGKILL**：强制杀死进程，本信号不能被阻塞和忽略。

- SIGTERM：与 SIGKILL 不同的是该信号可以被阻塞和处理。通常用来要求程序自己正常退出。

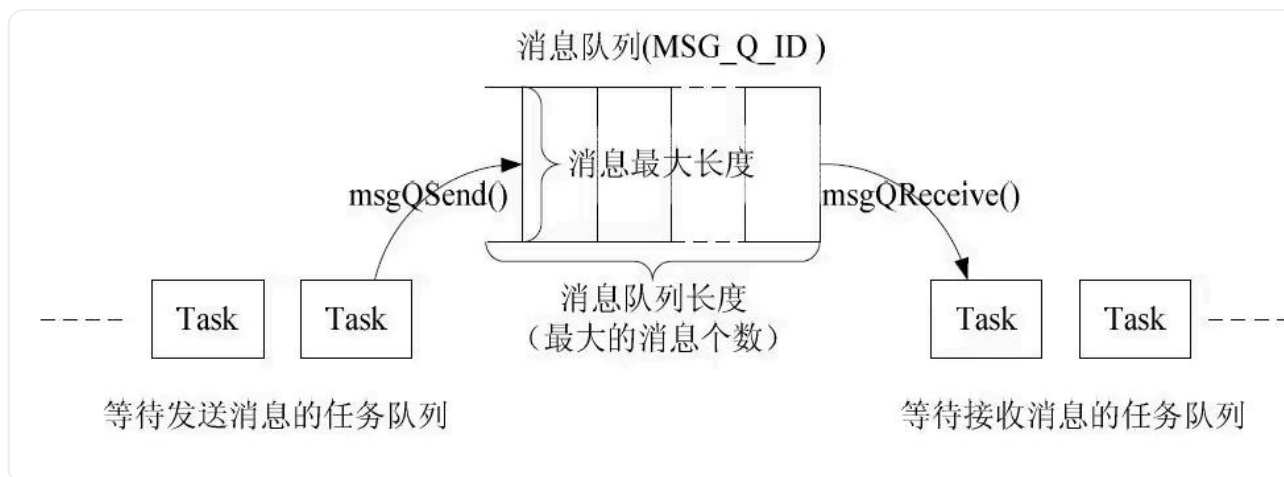
简单说说消息队列：

消息队列是保存在内核中的消息链表，按照消息的类型进行消息传递，具有较高的可靠性和稳定性。



编程十万问：消息队列

缺点：消息体有一个最大长度的限制，不适合比较大的数据传输；存在用户态与内核态之间的数据拷贝开销。

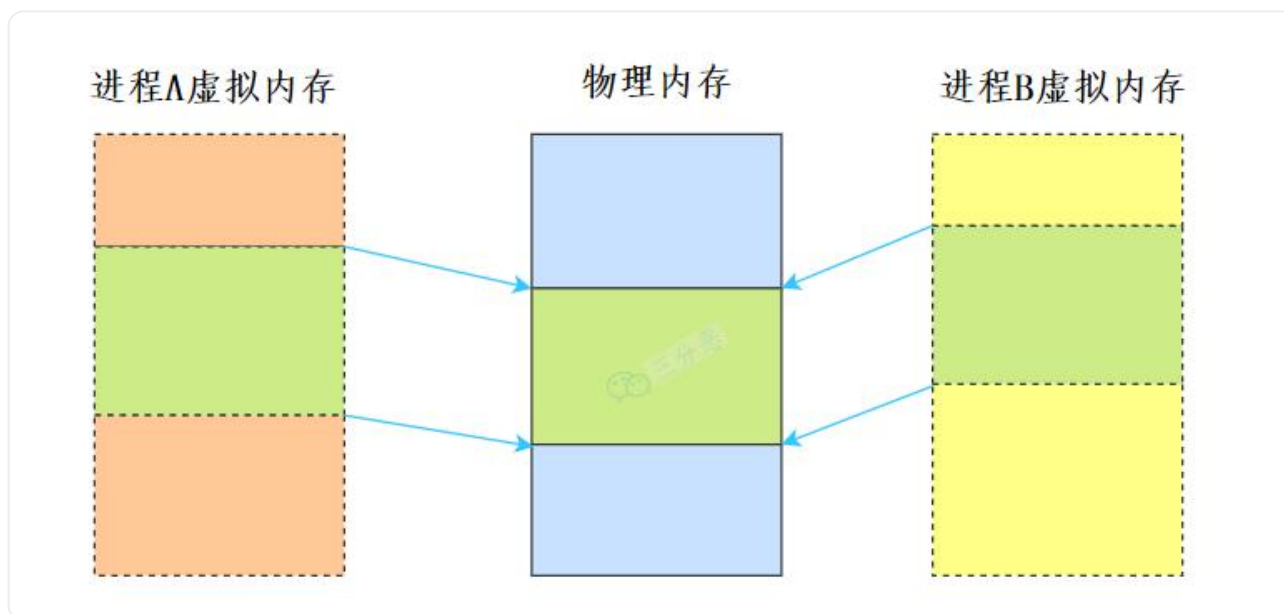


编程十万问：消息队列

简单说说共享内存：

允许两个或多个进程共享一个给定的内存区，一个进程写入的东西，其他进程马上就能看到。

共享内存是最快的进程间通信方式，它是针对其他进程间通信方式运行效率低而专门设计的。

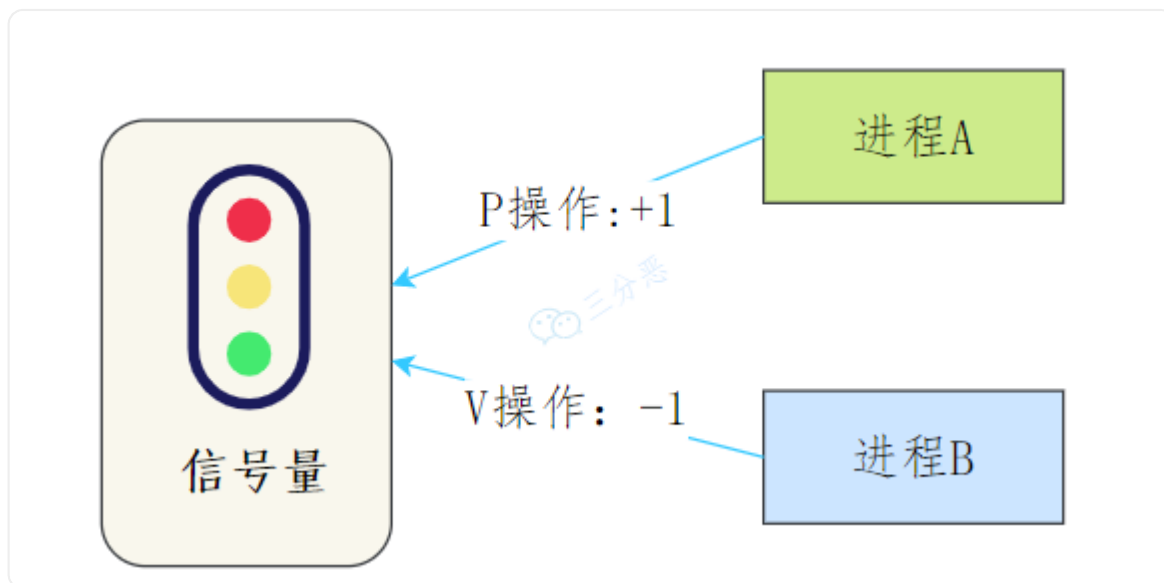


三分恶面渣逆袭：共享内存

缺点：当多进程竞争同一个共享资源时，会造成数据错乱的问题。

简单说说信号量：

信号量可以理解成红绿灯，红灯停（信号量为零），绿灯行（信号量非零）。它本质上是一个计数器，用来控制对共享资源的访问数量。

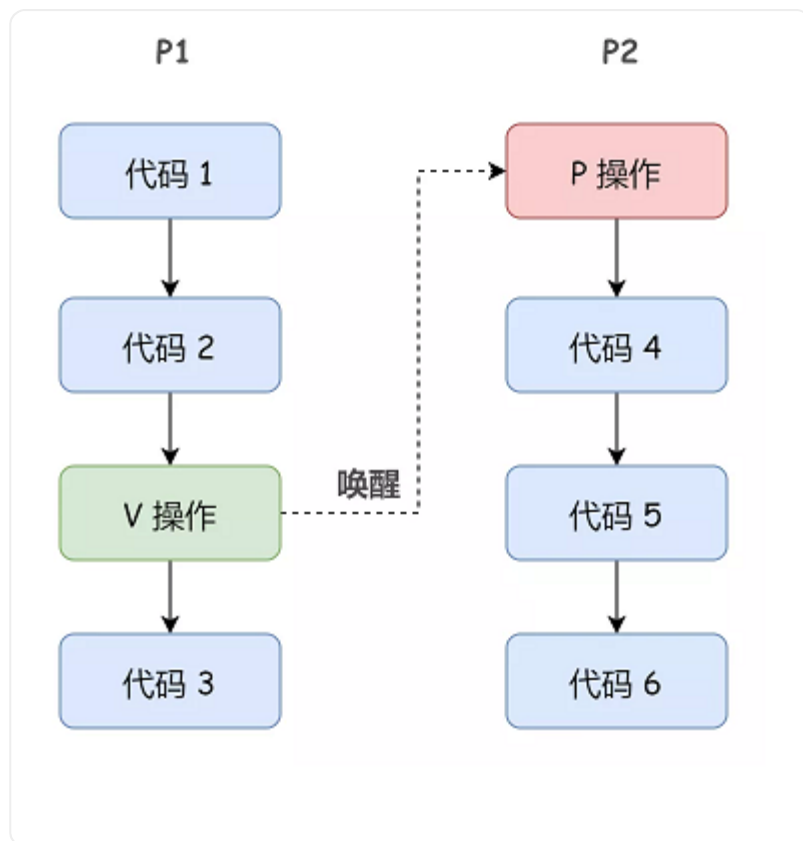


三分恶面渣逆袭：信号量

它常作为一种锁机制，防止某进程正在访问共享资源时，其他进程也访问该资源。Java 中的 [java.util.concurrent.Semaphore](https://javabetter.cn/thread/CountDownLatch.html#semaphore) 类 (<https://javabetter.cn/thread/CountDownLatch.html#semaphore>) 就实现了类似的功能。

控制信号量的方式有两种原子操作：

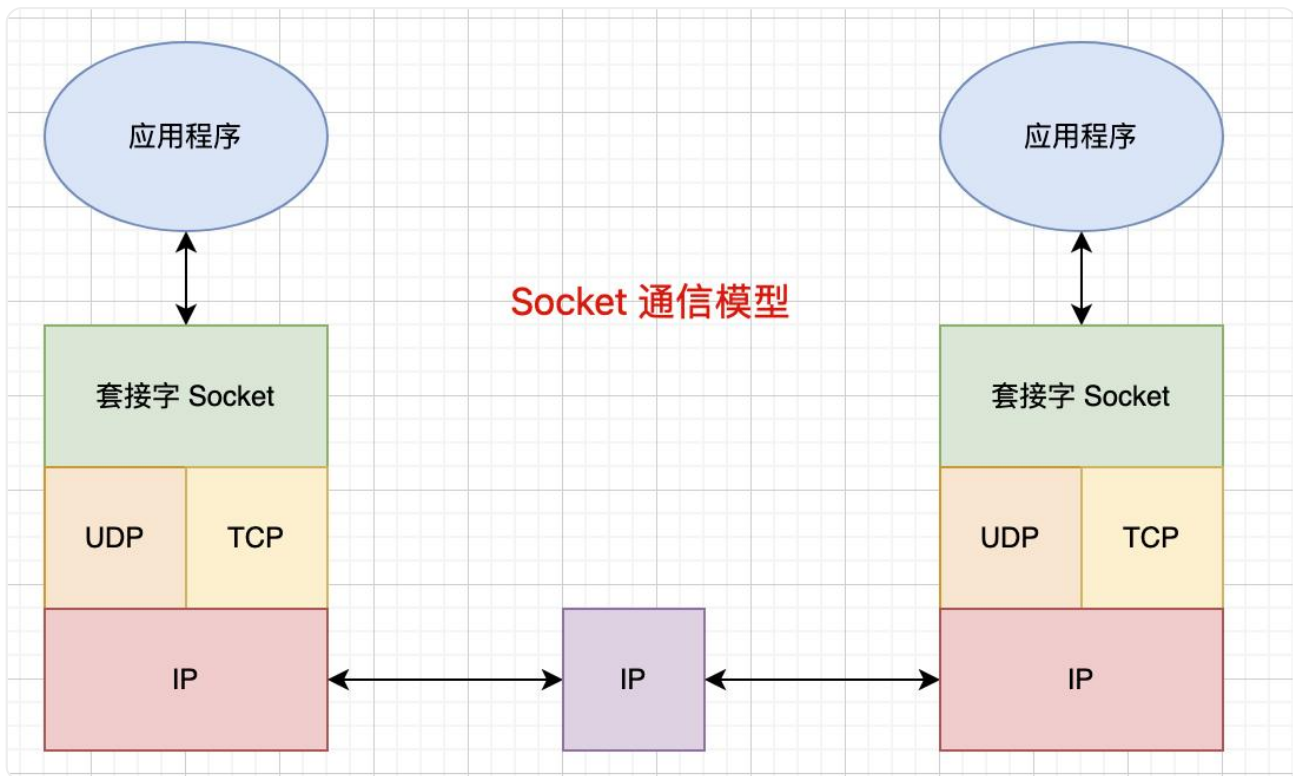
- 一个是 **P 操作**（wait，减操作），当进程希望获取资源时，它会执行 P 操作。如果信号量的值大于 0，表示有资源可用，信号量的值减 1，进程继续执行。如果信号量的值为 0，表示没有可用资源，进程进入等待状态，直到信号量的值变为大于 0。
- 另一个是 **V 操作**（signal，加操作），当进程释放资源时，它会执行 V 操作，信号量的值加 1。如果有其他进程因为等待该资源而被阻塞，这时会唤醒其中一个进程。



编程十万问：信号量

简单说说套接字 Socket：

这个和 Java 中的 Socket 很相似，提供网络通信的端点，可以让不同机器上运行的进程之间进行双向通信。



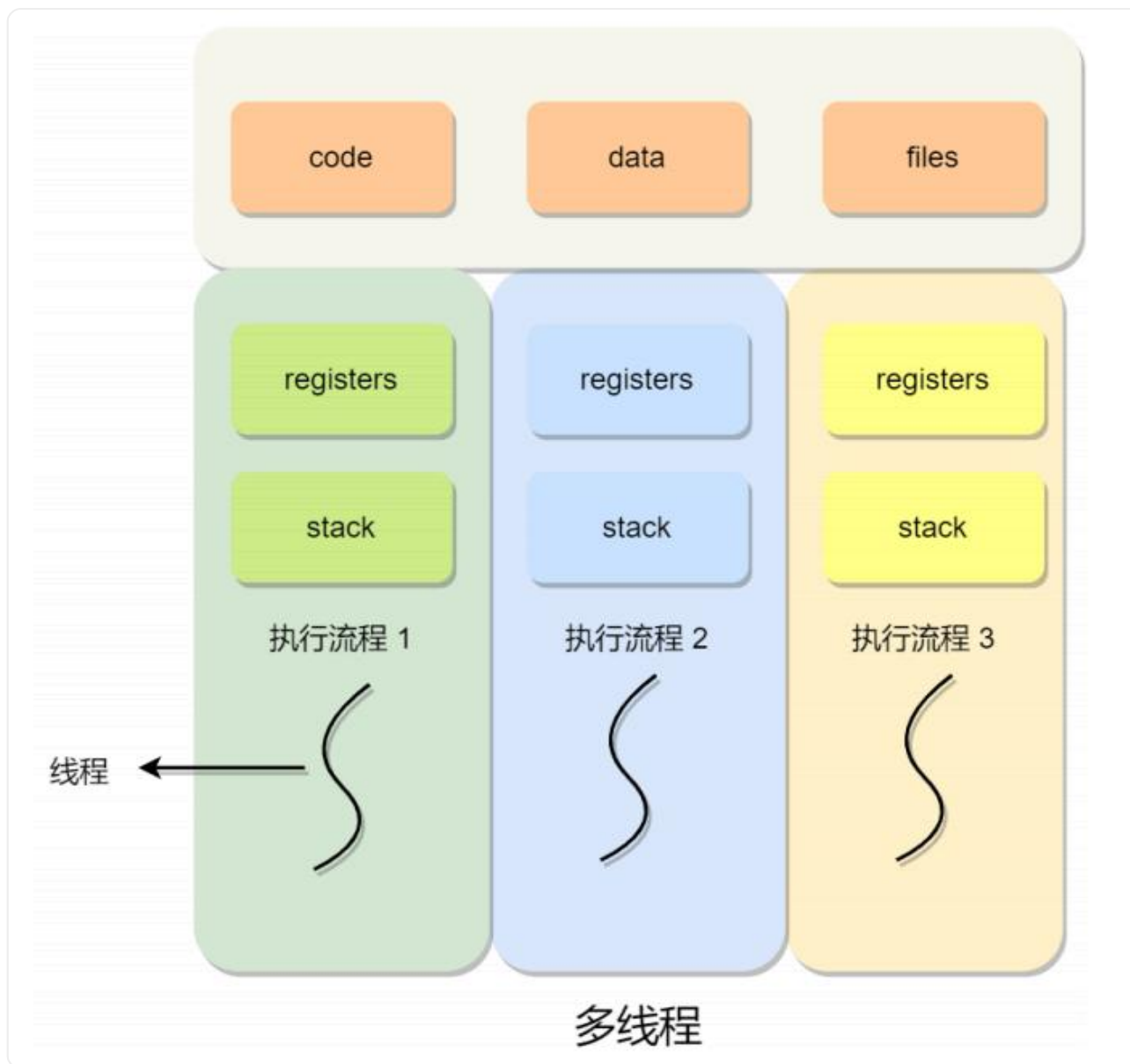
1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的华为一面原题：说一下进程的通信机制
2. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的字节跳动商业化一面的原题：进程和线程区别，线程共享内存和进程共享内存的区别
3. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的字节跳动商业化一面的原题：进程间如何通信
4. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的华为面经同学 6 Java 通用软件开发一面面试原题：说说你对 JVM 调优的了解
5. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的美团同学 2 优选物流调度技术 2 面面试原题：进程间的通信方式，代码使用匿名管道使两个进程通信

13、进程和线程的联系和区别？

线程和进程的联系：

线程是进程当中的一条执行流程。

同一个进程内多个线程之间可以共享代码段、数据段、打开的文件等资源，但每个线程各自都有一套独立的寄存器和栈，这样可以确保线程的控制流是相对独立的。



多线程-来源参考[3]

线程与进程的比较如下：

- 调度：**进程是资源（包括内存、打开的文件等）分配的单位，线程是 CPU 调度的单位；**
- 资源：进程拥有一个完整的资源平台，而线程只独享必不可少的资源，如寄存器和栈；
- 拥有资源：线程同样具有就绪、阻塞、执行三种基本状态，同样具有状态之间的转换关系；
- 系统开销：线程能减少并发执行的时间和空间开销——创建或撤销进程时，系统都要为之分配或回收系统资源，如内存空间，I/O 设备等，OS 所付出的开销显著大于在创建或撤销线程时的开销，进程切换的开销也远大于线程切换的开销。

14、线程上下文切换了解吗？

这还得看线程是不是属于同一个进程：

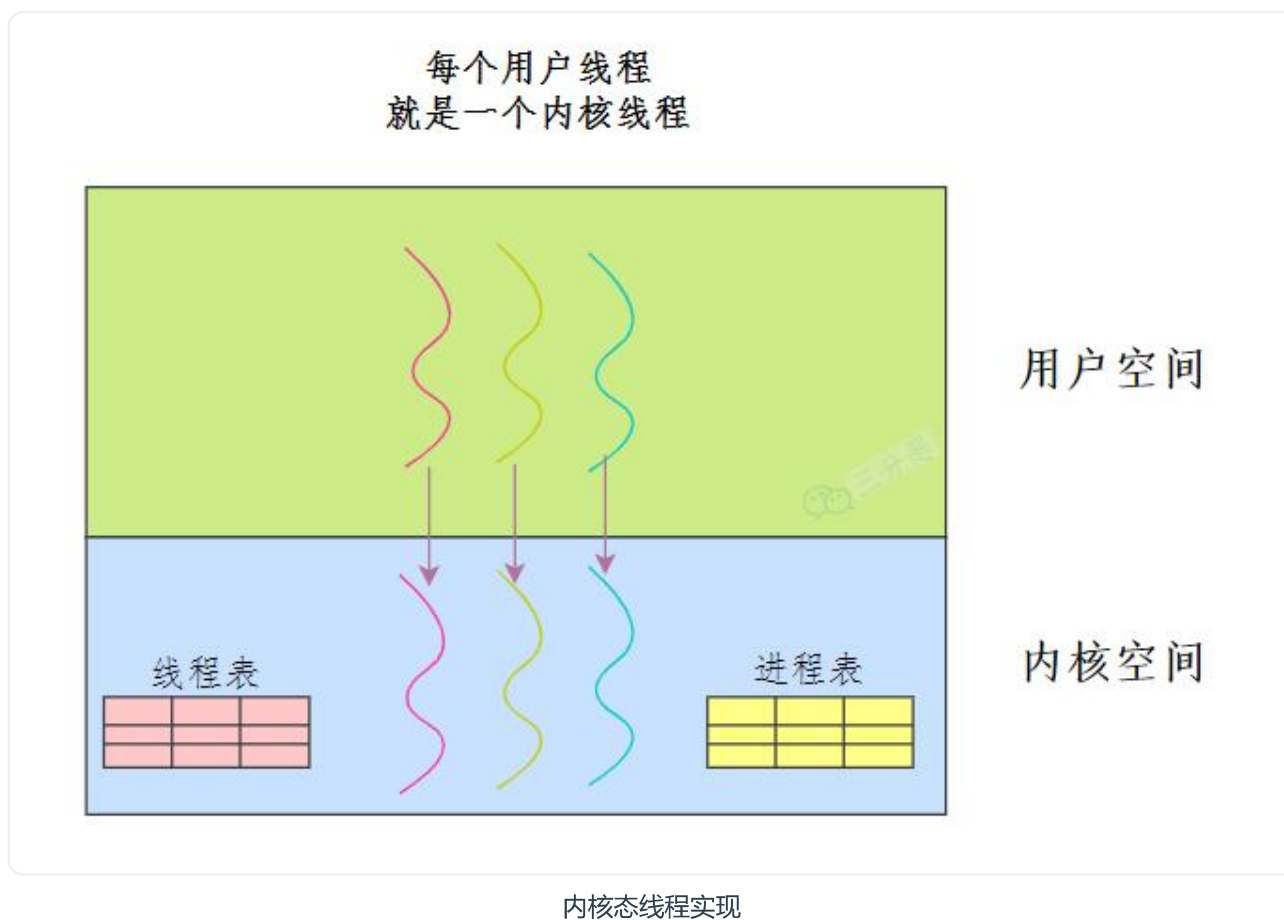
- 当两个线程不是属于同一个进程，则切换的过程就跟进程上下文切换一样；
- 当两个线程是属于同一个进程，因为虚拟内存是共享的，所以在切换时，虚拟内存这些资源就保持不动，只需要切换线程的私有数据、寄存器等不共享的数据；

所以，线程的上下文切换相比进程，开销要小很多。

15、线程有哪些实现方式？

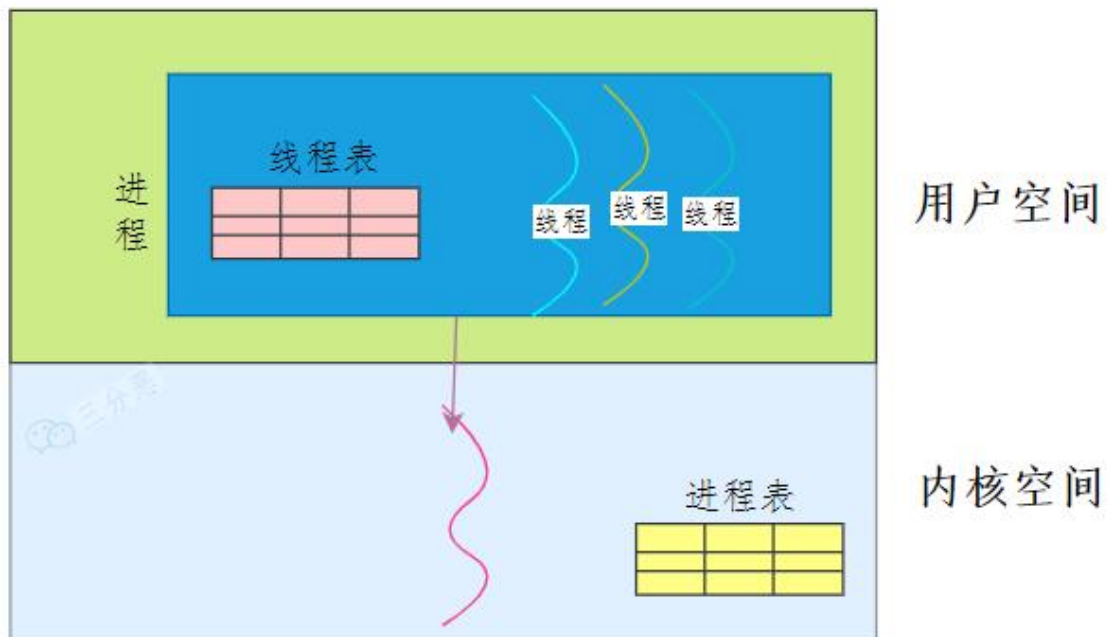
主要有三种线程的实现方式：

- **内核态线程实现**：在内核空间实现的线程，由内核直接管理直接管理线程。



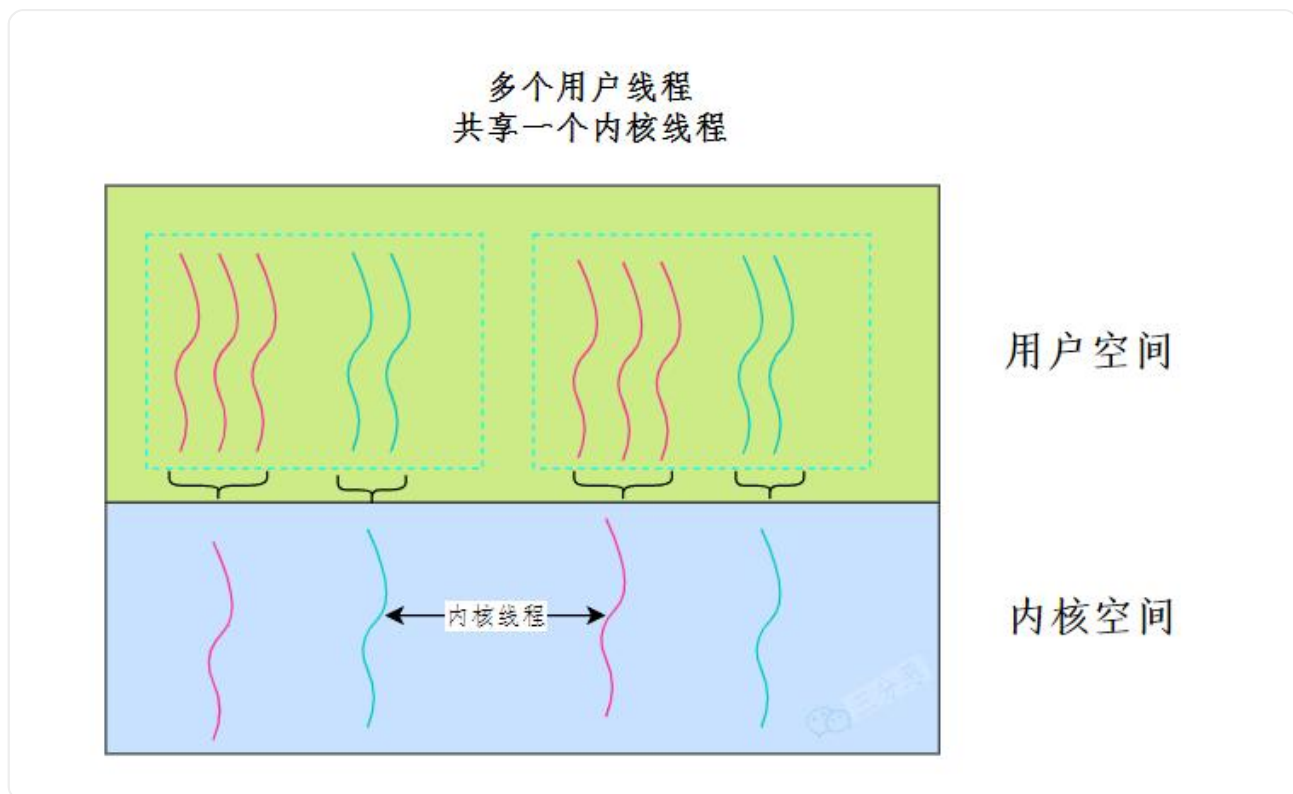
- **用户态线程实现**：在用户空间实现线程，不需要内核的参与，内核对线程无感知。

操作系统只看到线程



用户态线程

- **混合线程实现：**现代操作系统基本都是将两种方式结合起来使用。用户态的执行系统负责进程内部线程在非阻塞时的切换；内核态的操作系统负责阻塞线程的切换。即我们同时实现内核态和用户态线程管理。其中内核态线程数量较少，而用户态线程数量较多。每个内核态线程可以服务一个或多个用户态线程。

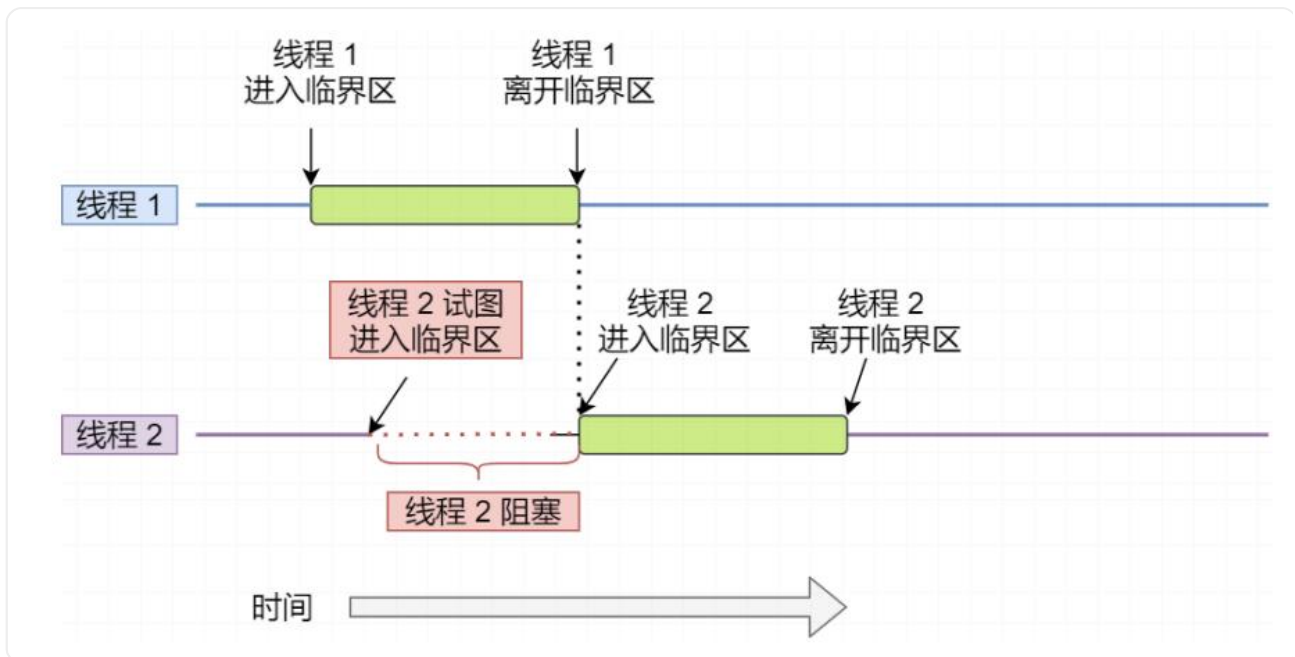


16、线程间如何同步？

同步解决的多线程操作共享资源的问题，目的是不管线程之间的执行如何穿插，最后的结果都是正确的。

我们前面知道线程和进程的关系：线程是进程当中的一条执行流程。所以说下面的一些同步机制不止针对线程，同样也可以针对进程。

临界区：我们把对共享资源访问的程序片段称为 临界区，我们希望这段代码是 互斥 的，保证在某时刻只能被一个线程执行，也就是说一个线程在临界区执行时，其它线程应该被阻止进入临界区。



临界区互斥-来源参考[3]

临界区不仅针对线程，同样针对进程。

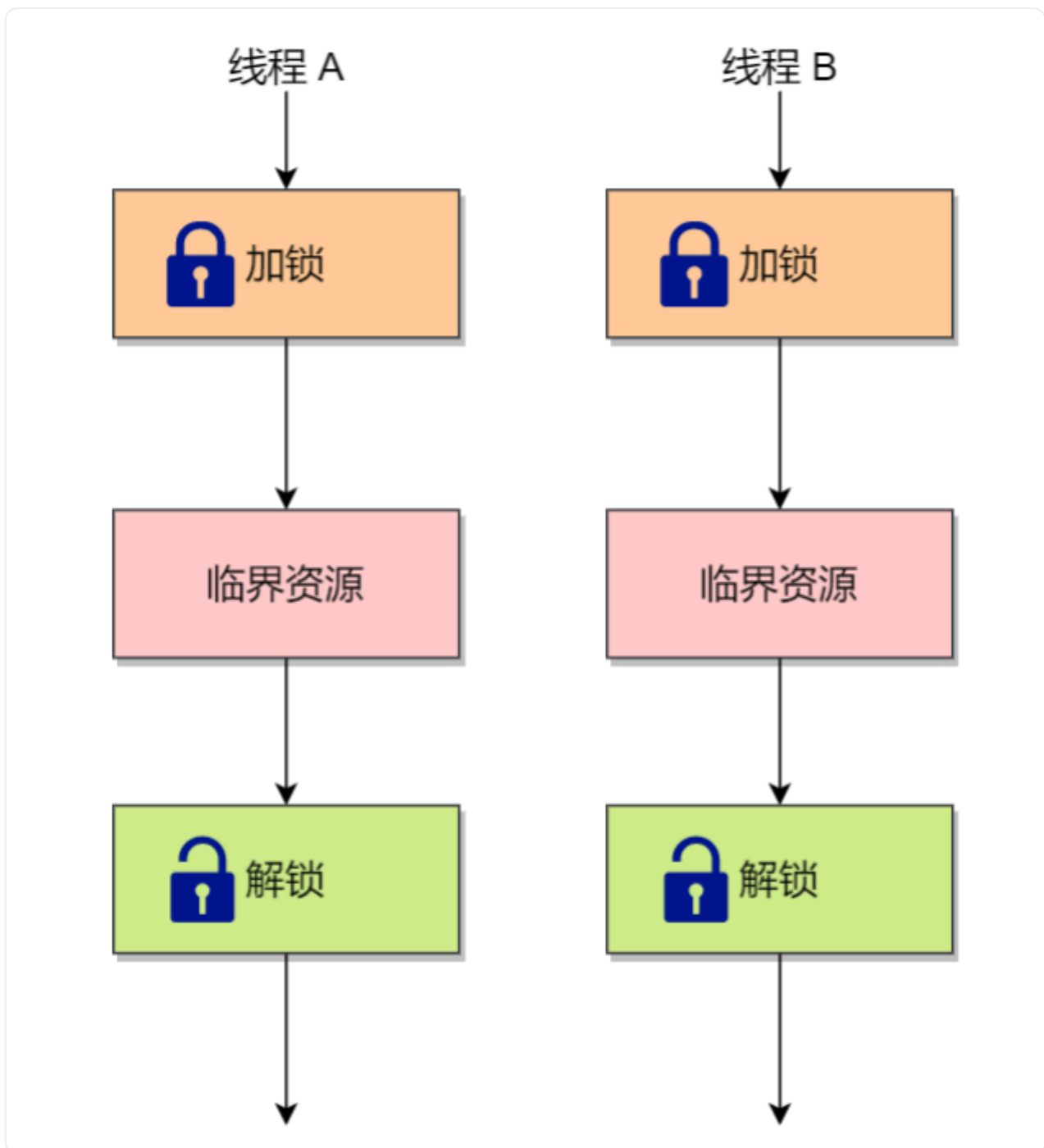
临界区同步的一些实现方式：

1、锁

使用加锁操作和解锁操作可以解决并发线程/进程的互斥问题。

任何想进入临界区的线程，必须先执行加锁操作。若加锁操作顺利通过，则线程可进入临界区；在完成对临界资源的访问后再执行解锁操作，以释放该临界资源。

加锁和解锁锁住的是什么呢？可以是临界区对象，也可以只是一个简单的互斥量，例如互斥量是 0 无锁，1 表示加锁。



加锁和解锁-来源参考[3]

根据锁的实现不同，可以分为 忙等待锁 和 无忙等待锁 。

忙等待锁 就是加锁失败的线程，会不断尝试获取锁，也被称为自旋锁，它会一直占用 CPU。

无忙等待锁 就是加锁失败的线程，会进入阻塞状态，放弃 CPU，等待被调度。

2、信号量

信号量是操作系统提供的一种协调共享资源访问的方法。

通常信号量表示资源的数量，对应的变量是一个整型（sem）变量。

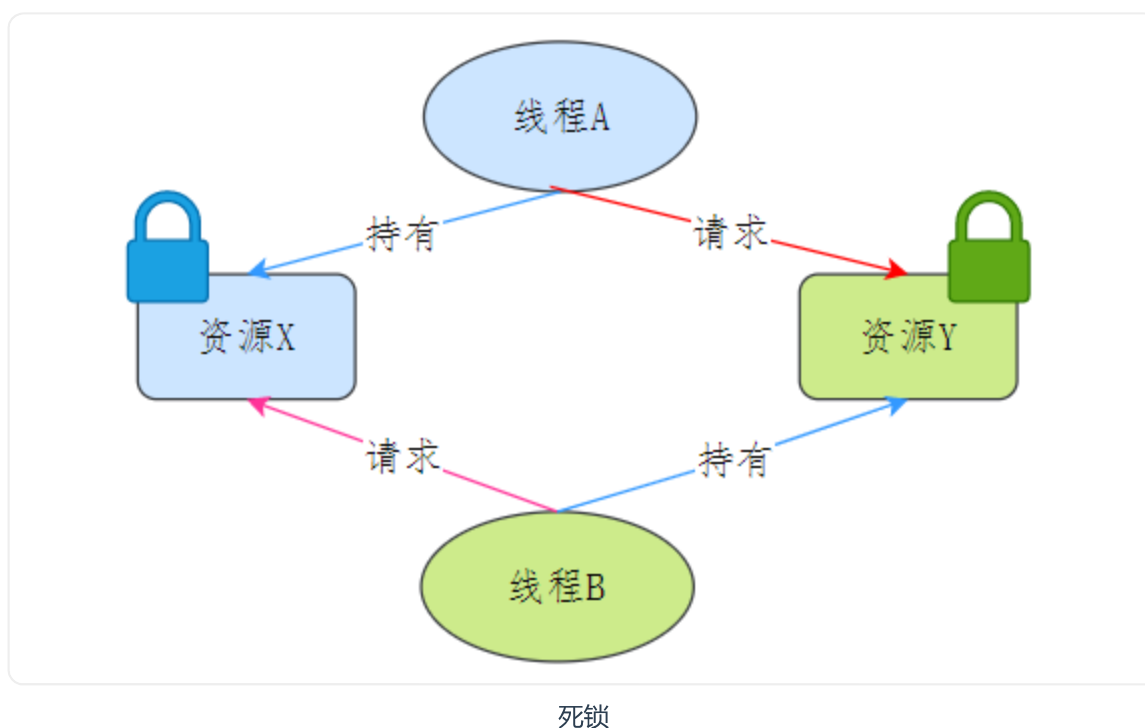
另外，还有两个原子操作的系统调用函数来控制信号量的，分别是：

- P操作：将 sem 减 1，相减后，如果 $\text{sem} < 0$ ，则进程/线程进入阻塞等待，否则继续，表明 P 操作可能会阻塞；
- V操作：将 sem 加 1，相加后，如果 $\text{sem} \leq 0$ ，唤醒一个等待中的进程/线程，表明 V 操作不会阻塞；

P 操作是用在进入临界区之前，V 操作是用在离开临界区之后，这两个操作是必须成对出现的。

17、什么是死锁？

在两个或者多个并发线程中，如果每个线程持有某种资源，而又等待其它线程释放它或它们现在保持着的资源，在未改变这种状态之前都不能向前推进，称这一组线程产生了死锁。通俗的讲就是两个或多个线程无限期的阻塞、相互等待的一种状态。



18、死锁产生有哪些条件？

死锁产生需要同时满足四个条件：

- **互斥条件**：指线程对已经获取到的资源进行它性使用，即该资源同时只由一个线程占用。如果此时还有其它线程请求获取获取该资源，则请求者只能等待，直至占有资源的线程释

放该资源。

- **请求并持有条件**：指一个线程已经持有了至少一个资源，但又提出了新的资源请求，而新资源已被其它线程占有，所以当前线程会被阻塞，但阻塞的同时并不释放自己已经获取的资源。
- **不可剥夺条件**：指线程获取到的资源在自己使用完之前不能被其它线程抢占，只有在自己使用完毕后才由自己释放该资源。
- **环路等待条件**：指在发生死锁时，必然存在一个线程——资源的环形链，即线程集合 {T0, T1, T2,....., Tn} 中 T0 正在等待 T1 占用的资源，T1 正在等待 T2 用的资源，..... Tn 在等待已被 T0 占用的资源。

19、如何避免死锁呢？

产生死锁的有四个必要条件：互斥条件、持有并等待条件、不可剥夺条件、环路等待条件。

避免死锁，破坏其中的一个就可以。

消除互斥条件

这个是无法实现，因为很多资源就是只能被一个线程占用，例如锁。

消除请求并持有条件

消除这个条件的办法很简单，就是一个线程一次请求其所需要的所有资源。

消除不可剥夺条件

占用部分资源的线程进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源，这样不可剥夺这个条件就破坏掉了。

消除环路等待条件

可以靠按序申请资源来预防。所谓按序申请，是指资源是有线性顺序的，申请的时候可以先生申请资源序号小的，再申请资源序号大的，这样线性化后就不存在环路了。

20、活锁和饥饿锁了解吗？

饥饿锁：

饥饿锁，这个饥饿指的是资源饥饿，某个线程一直等不到它所需要的资源，从而无法向前推进，就像一个人因为饥饿无法成长。

活锁：

在活锁状态下，处于活锁线程组里的线程状态可以改变，但是整个活锁组的线程无法推进。

活锁可以用两个人过一条很窄的小桥来比喻：为了让对方先过，两个人都往旁边让，但两个人总是让到同一边。这样，虽然两个人的状态一直在变化，但却都无法往前推进。

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer) (<https://github.com/itwanger/toBeBetterJavaer>)》第一版 PDF 终于来了！包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等，共计 32 万余字，500+张手绘图，可以说是通俗易懂、风趣幽默.....详情戳：[太赞了，GitHub 上标星 10000+ 的 Java 教程](https://javabetter.cn/overview/) (<https://javabetter.cn/overview/>)

微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二，回复 **222** 即可免费领取。



内存管理

21、什么是虚拟内存？

我们实际的物理内存主要是主存，但是物理主存空间有限，所以一般现代操作系统都会想办法把一部分内存块放到磁盘中，用到的时候再装入主存，但是对用户程序而言，是不需要注意实际的物理内存的，为什么呢？因为有 **虚拟内存** 的机制。

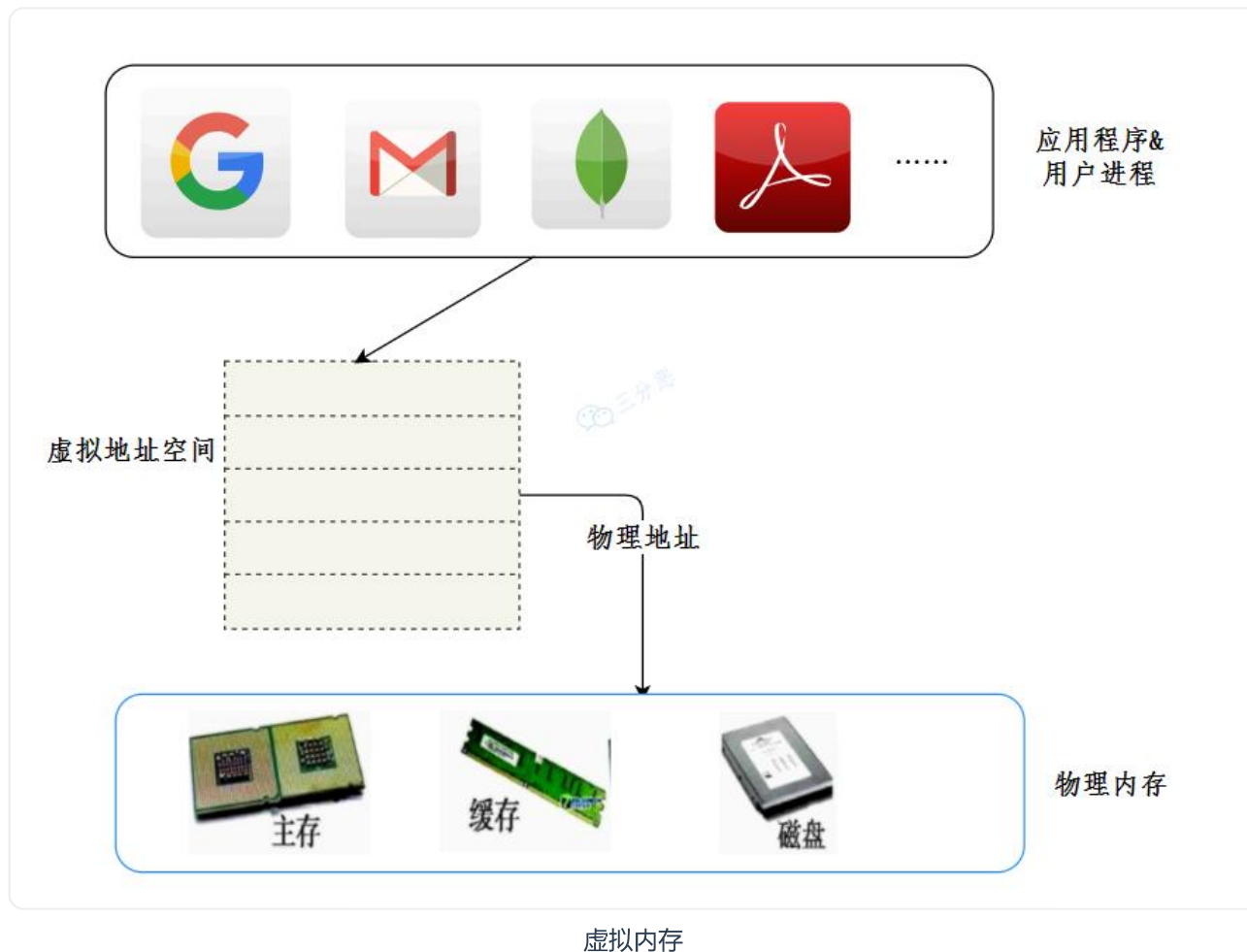
简单说，**虚拟内存**是操作系统提供的一种机制，将不同进程的**虚拟地址**和不同内存的**物理地址**映射起来。

每个进程都有自己独立的地址空间，再由操作系统映射到实际的物理内存。

于是，这里就引出了两种地址的概念：

程序所使用的内存地址叫做**虚拟内存地址**（*Virtual Memory Address*）

实际存在硬件里面的空间地址叫**物理内存地址**（*Physical Memory Address*）。



22、什么是内存分段？

程序是由若干个逻辑分段组成的，如可由代码分段、数据分段、栈段、堆段组成。不同的段是有不同的属性的，所以就用分段（Segmentation）的形式把这些段分离出来。

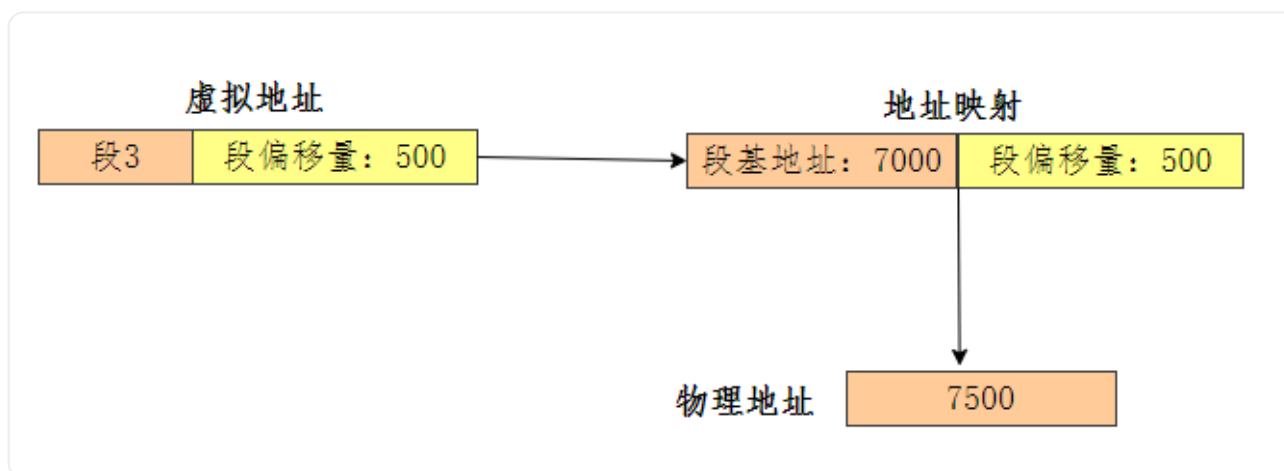
分段机制下的虚拟地址由两部分组成，**段号**和**段内偏移量**。

虚拟地址和物理地址通过段表映射，段表主要包括**段号**、段的界限。



虚拟地址、段表、物理地址

我们来看一个映射，虚拟地址：段 3、段偏移量 500 ----> 段基地址 7000+段偏移量 500 - ----> 物理地址：8700+。

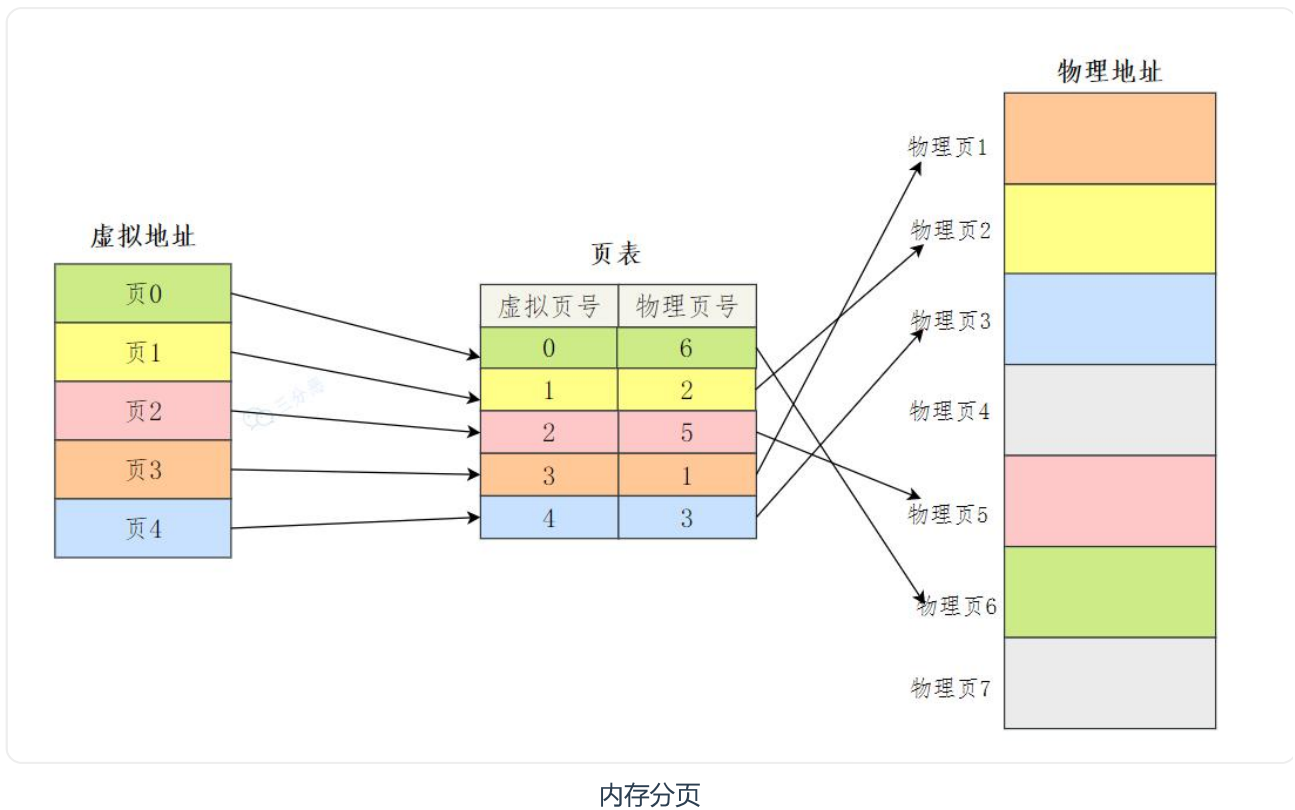


段虚拟地址映射

23、什么是内存分页？

分页是把整个虚拟和物理内存空间切成一段段固定尺寸的大小。这样一个连续并且尺寸固定的内存空间，我们叫页（Page）。在 Linux 下，每一页的大小为 4KB。

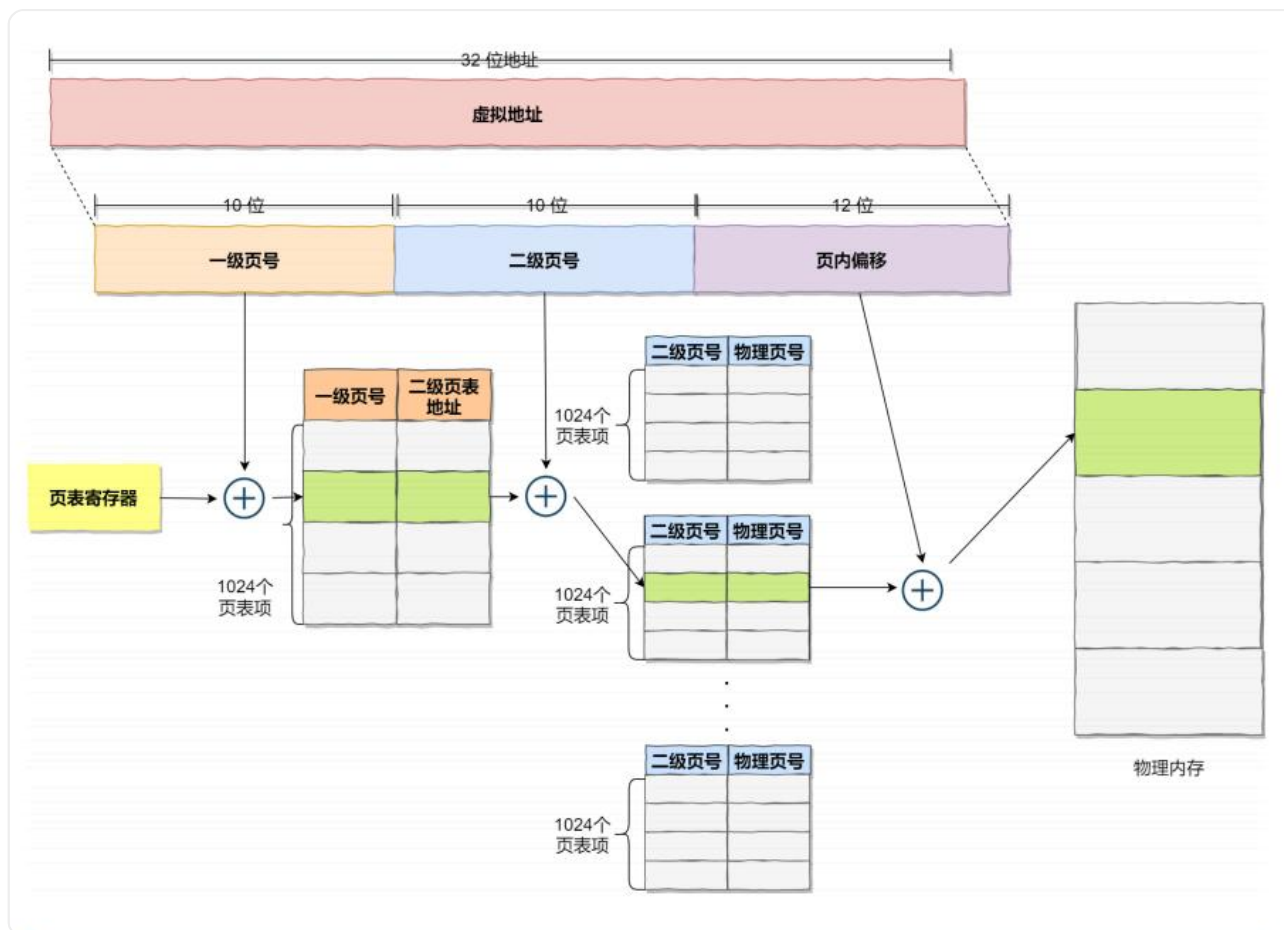
访问分页系统中内存数据需要两次的内存访问：一次是从内存中访问页表，从中找到指定的物理页号，加上页内偏移得到实际物理地址，第二次就是根据第一次得到的物理地址访问内存取出数据。



24、多级页表知道吗？

推荐阅读：[操作系统导论：多级页表 \(https://taifua.com/ostep-vm-smalltables.html\)](https://taifua.com/ostep-vm-smalltables.html)

多级页表 (Multilevel Page Table) 是一种内存管理技术，用于在虚拟内存系统中高效地管理和转换虚拟地址到物理地址。它通过分层结构减少页表所需的内存开销，以解决单级页表在大地址空间中的效率问题。



三分恶面渣逆袭：多级页表示意图

在虚拟内存系统中，虚拟地址需要转换为物理地址。页表是实现这种转换的关键数据结构。对于 32 位系统，一个进程的地址空间可以达到 4 GB，如果使用单级页表，每个页表条目（PTE）占用 4 字节，则需要 4 MB 的内存来存储页表。然而，许多进程只使用其中的一小部分地址空间，导致单级页表的内存浪费。

多级页表通过将单级页表拆分为多个层级，减少了内存浪费。以两级页表为例：

- 一级页表（页目录）：存储二级页表的地址。每个页目录条目（PDE）指向一个二级页表。
- 二级页表（页表）：存储实际的页框地址。每个页表条目（PTE）指向一个物理页框。

虚拟地址分为多个部分，每一部分用于索引相应层级的页表。例如，对于一个 32 位地址和 4 KB 页大小的两级页表：

- 高 10 位：一级页表索引（页目录索引）。
- 中 10 位：二级页表索引（页表索引）。
- 低 12 位：页内偏移。

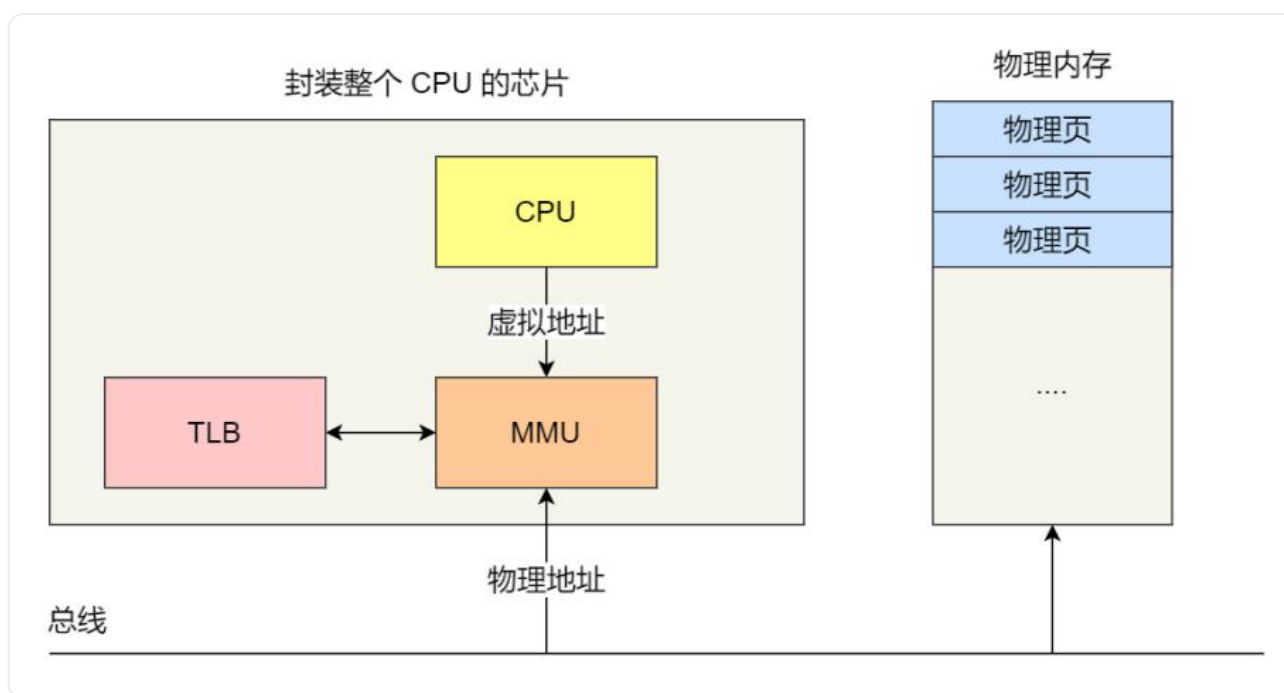
1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的

得物面经同学 1 面试原题：多级页表

25、什么是快表？

同样利用了 局部性原理，即在一段时间内，整个程序的执行仅限于程序中的某一部分。相应地，执行所访问的存储空间也局限于某个内存区域。

利用这一特性，把最常访问的几个页表项存储到访问速度更快的硬件，于是计算机科学家们，就在 CPU 芯片中，加入了一个专门存放程序最常访问的页表项的 Cache，这个 Cache 就是 TLB（*Translation Lookaside Buffer*），通常称为页表缓存、转址旁路缓存、快表等。



TLB示意图-来源参考[3]

26、分页和分段有什么区别？

- 段是信息的逻辑单位，它是根据用户的需要划分的，因此段对用户是可见的；页是信息的物理单位，是为了管理主存的方便而划分的，对用户是透明的。
- 段的大小不固定，有它所完成的功能决定；页的大小固定，由系统决定
- 段向用户提供二维地址空间；页向用户提供的是一维地址空间
- 段是信息的逻辑单位，便于存储保护和信息的共享，页的保护和共享受到限制。

27、什么是交换空间？

操作系统把物理内存(Physical RAM)分成一块一块的小内存，每一块内存被称为页(page)。当内存资源不足时，Linux 把某些页的内容转移至磁盘上的一块空间上，以释放内存空间。磁盘上的那块空间叫做交换空间(swap space)，而这一过程被称为交换(swapping)。物理内存和交换空间的总容量就是虚拟内存的可用容量。

用途：

- 物理内存不足时一些不常用的页可以被交换出去，腾给系统。
- 程序启动时很多内存页被用来初始化，之后便不再需要，可以交换出去。

33、什么是缺页中断？（补充）

2024 年 03 月 29 日增补

缺页中断（Page Fault）是虚拟内存管理的一个重要概念。当一个程序访问的页（页面）不在物理内存中时，就会发生缺页中断。操作系统需要从磁盘上的交换区（或页面文件）中将缺失的页调入内存。

举个例子，你正在一间图书馆（内存）里查找一本特定的书（数据/程序页），图书馆的书架（内存空间）能放的书是有限的。现在，如果你找的那本书正好在书架上，那太好了，直接拿来阅读（内存命中）。

但如果书架上没有（缺页），你需要先去找图书管理员。

图书管理员（操作系统）注意到书架上缺了这本书，然后去仓库里帮你找（缺页中断）。找到书之后，管理员发现书架已经满了，需要先从书架上拿掉一本书（页面置换算法决定哪本书被拿掉），然后把新找到的书放上去，最后把书递给你。

这个过程中，“去仓库找书并换回来”的这一过程就像是发生了缺页中断，而决定哪本书被移出书架以腾出位置放新书的规则，就是页面置换算法在做的事情。

这么做的目的是尽量确保你常读的书都能在书架（内存）上直接找到，避免每次都去仓库（硬盘）搜寻，因为去仓库找书的过程比较耗时。

1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的字节跳动面经同学 9 飞书后端技术一面面试原题：操作系统缺页中断，页面置换算法

28、页面置换算法有哪些？

推荐阅读：[页面置换算法详解 \(https://www.cnblogs.com/Leophen/p/11397699.html\)](https://www.cnblogs.com/Leophen/p/11397699.html)

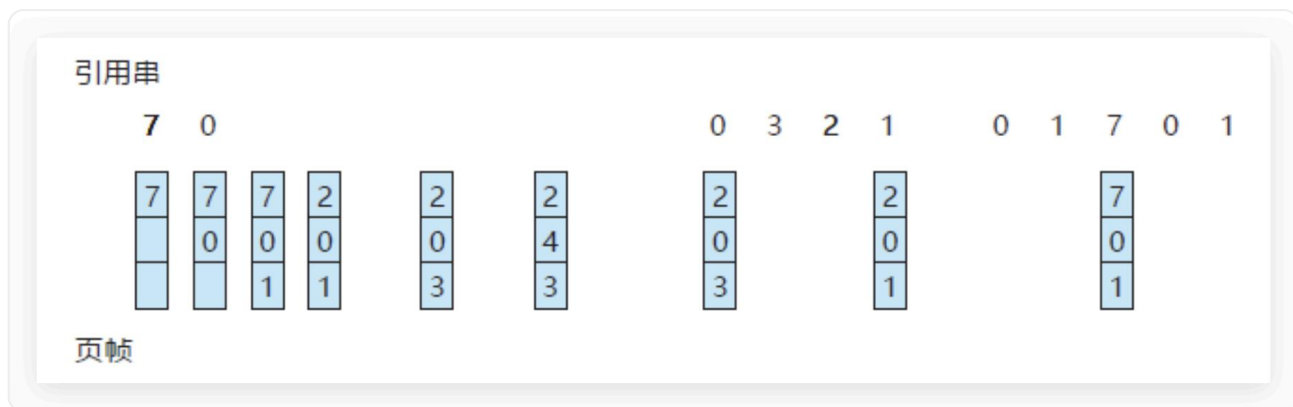
页面置换算法的目标是最小化缺页中断的次数，常见的页面置换算法有最佳页面置换算法（*OPT*）、先进先出置换算法（*FIFO*）、最近最久未使用的置换算法（*LRU*）和时钟页面置换算法等。



三分恶面渣逆袭：常见页面置换算法

①、最佳页面置换算法

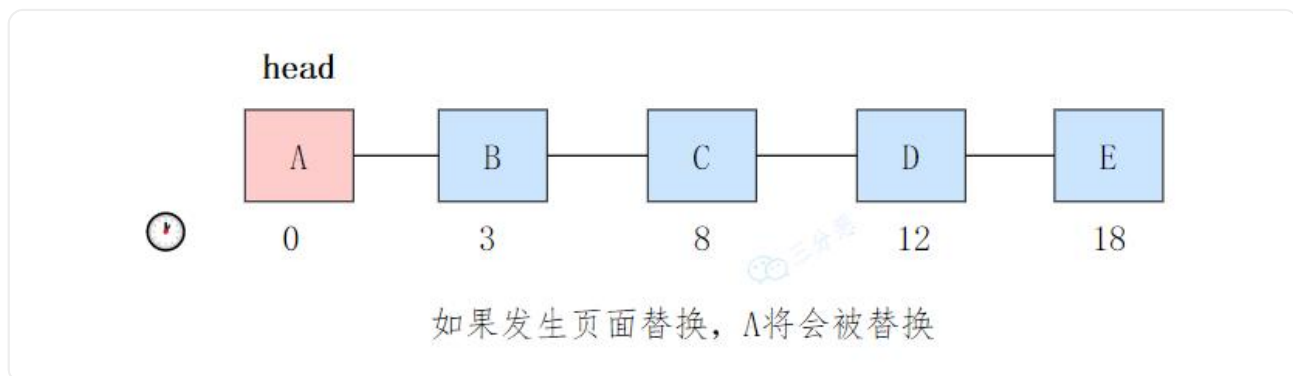
基本思路是，淘汰以后不会使用的页面。这是理论上的最佳算法，因为它可以保证最低的缺页率。但在实际应用中，由于无法预知未来的访问模式，*OPT* 通常无法实现。



Leophen: OPT

②、先进先出置换算法

基本思路是，优先淘汰最早进入内存的页面。FIFO 算法维护一个队列，新来的页面加入队尾，当发生页面置换时，队头的页面（即最早进入内存的页面）被移出。

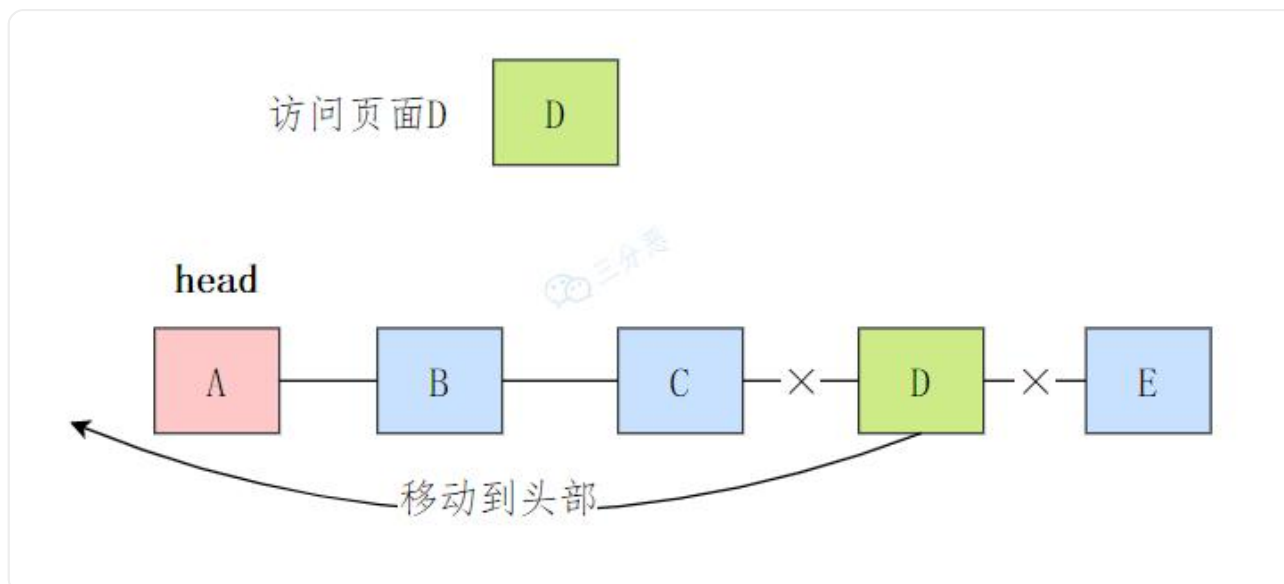


三分恶面渣逆袭：按照进入内存早晚构建的页面链表

③、最近最久未使用的置换算法

基本思路是，淘汰最近没有使用的页面。LRU 算法根据页面的访问历史来进行置换，最长时间未被访问的页面将被置换出去。

相对更接近最优算法的效果，因为最近未使用的页面可能在将来也不会被使用。但 LRU 算法的实现需要跟踪页面的访问历史，可能会增加系统的开销。



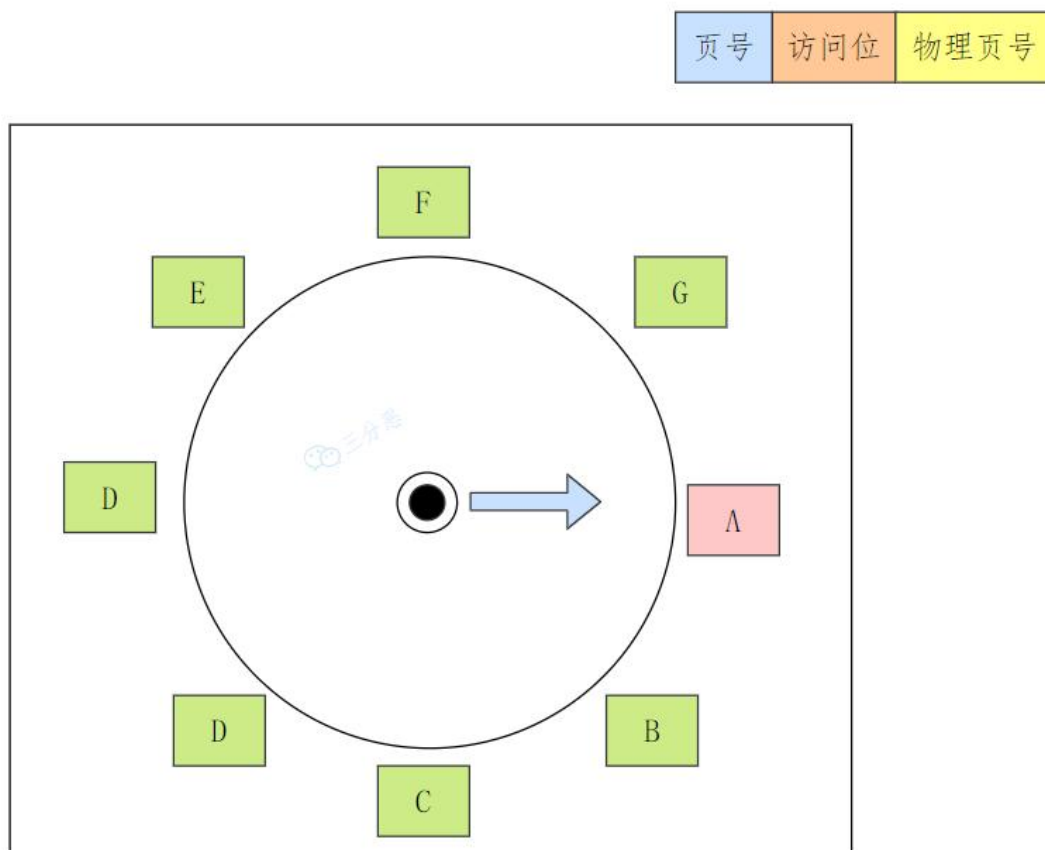
三分恶面渣逆袭：LRU实现

④、时钟页面置换算法

时钟算法是 LRU 的一种近似和实现简单的形式。它通过一个循环列表（类似时钟的指针）遍历页面，每个页面有一个使用位，当页面被访问时，使用位设置为 1。

当需要页面置换时，时钟指针会顺时针移动，直到找到使用位为 0 的页面进行置换。这个过程类似于给每个页面一个二次机会。算法执行时，会先将使用位从 1 清零，如果该页面再次被访问，它的使用位再次被设置为 1。

链表节点



三分恶面渣逆袭：时钟页面置换算法

⑤、最不常用置换算法

根据页面被访问的频率进行置换，访问次数最少的页面最先被置换。实现较为复杂，需要记录每个页面的访问频率。

1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的字节跳动面经同学 9 飞书后端技术一面面试原题：操作系统缺页中断，页面置换算法

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer) (<https://github.com/itwanger/toBeBetterJavaer>)》第一版 PDF 终于来了！包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等，共计 32 万余字，500+张手绘图，可以说是通俗易懂、风趣幽默.....详情戳：[太赞了，GitHub 上标星 10000+ 的 Java 教程](https://javabetter.cn/overview/) (<https://javabetter.cn/overview/>)

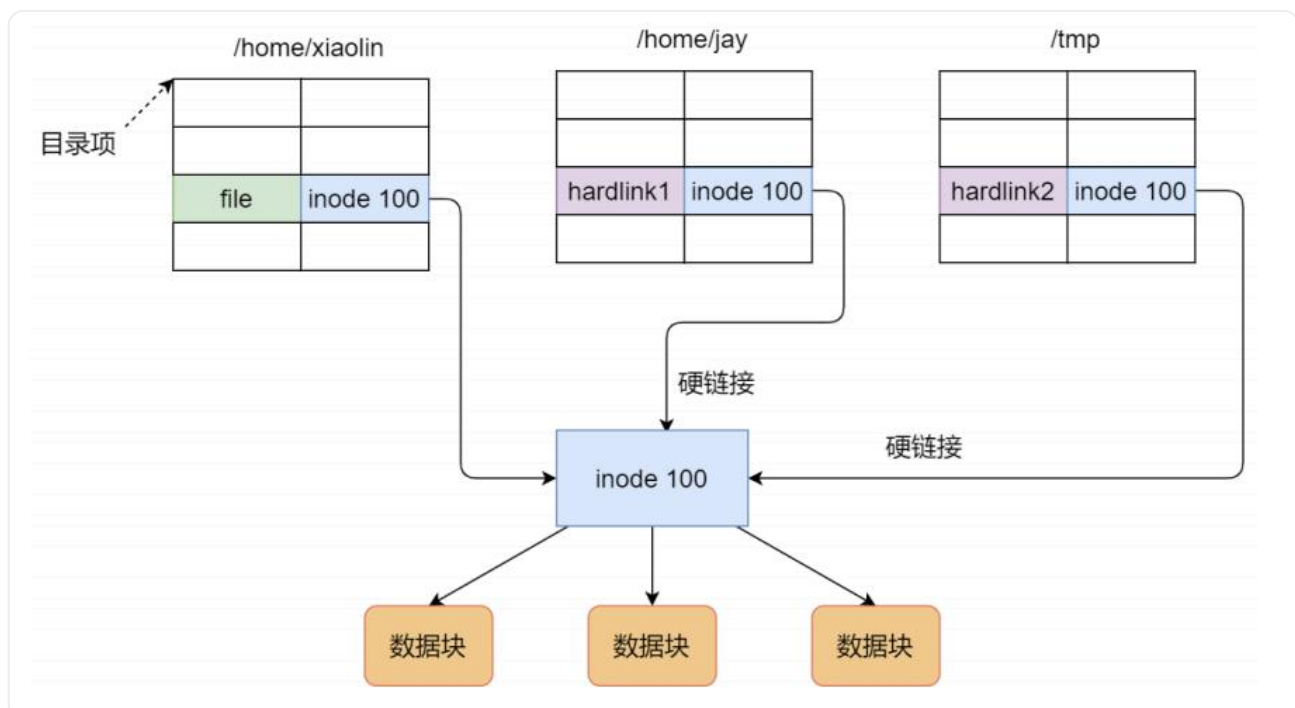
微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二，回复 **222** 即可免费领取。



文件

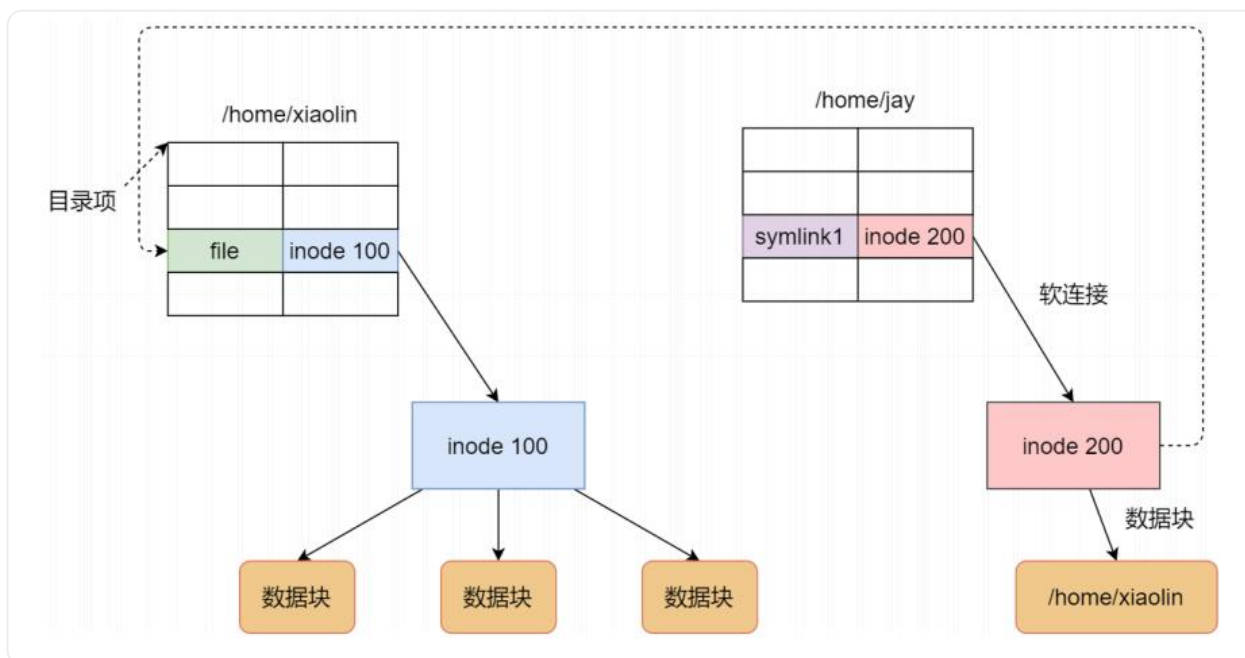
29、硬链接和软链接有什么区别？

- 硬链接就是在目录下创建一个条目，记录着文件名与 inode 编号，这个 inode 就是源文件的 inode。删除任意一个条目，文件还是存在，只要引用数量不为 0。但是硬链接有限制，它不能跨越文件系统，也不能对目录进行链接。



硬链接-来源参考[3]

- 软链接相当于重新创建一个文件，这个文件有**独立的 inode**，但是这个文件的**内容是另外一个文件的路径**，所以访问软链接的时候，实际上相当于访问到了另外一个文件，所以**软链接是可以跨文件系统的**，甚至**目标文件被删除了**，**链接文件还是在的**，只不过**打不开指向的文件了而已**。

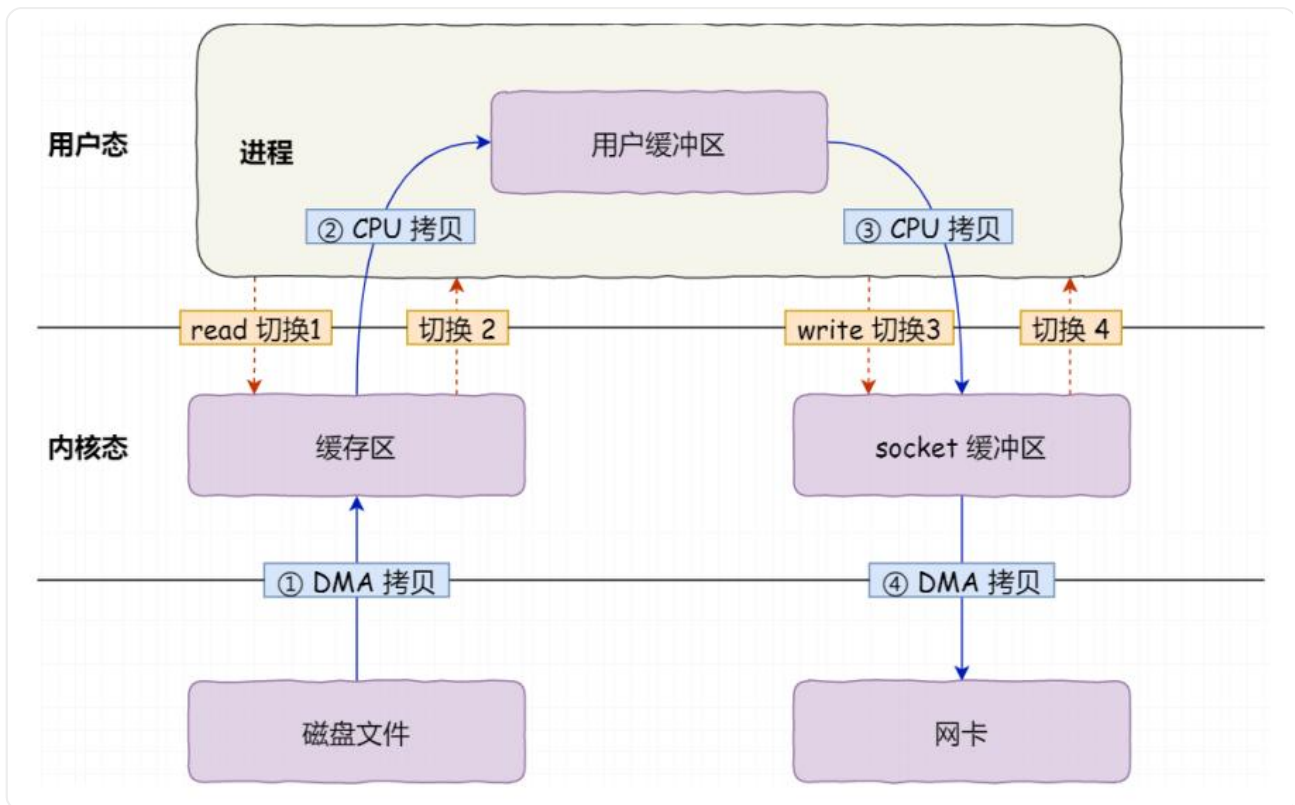


软链接-来源参考[3]

IO

30、零拷贝了解吗？

假如需要文件传输，使用传统 I/O，数据读取和写入是用户空间到内核空间来回赋值，而内核空间的数据是通过操作系统的 I/O 接口从磁盘读取或者写入，这期间发生了多次用户态和内核态的上下文切换，以及多次数据拷贝。



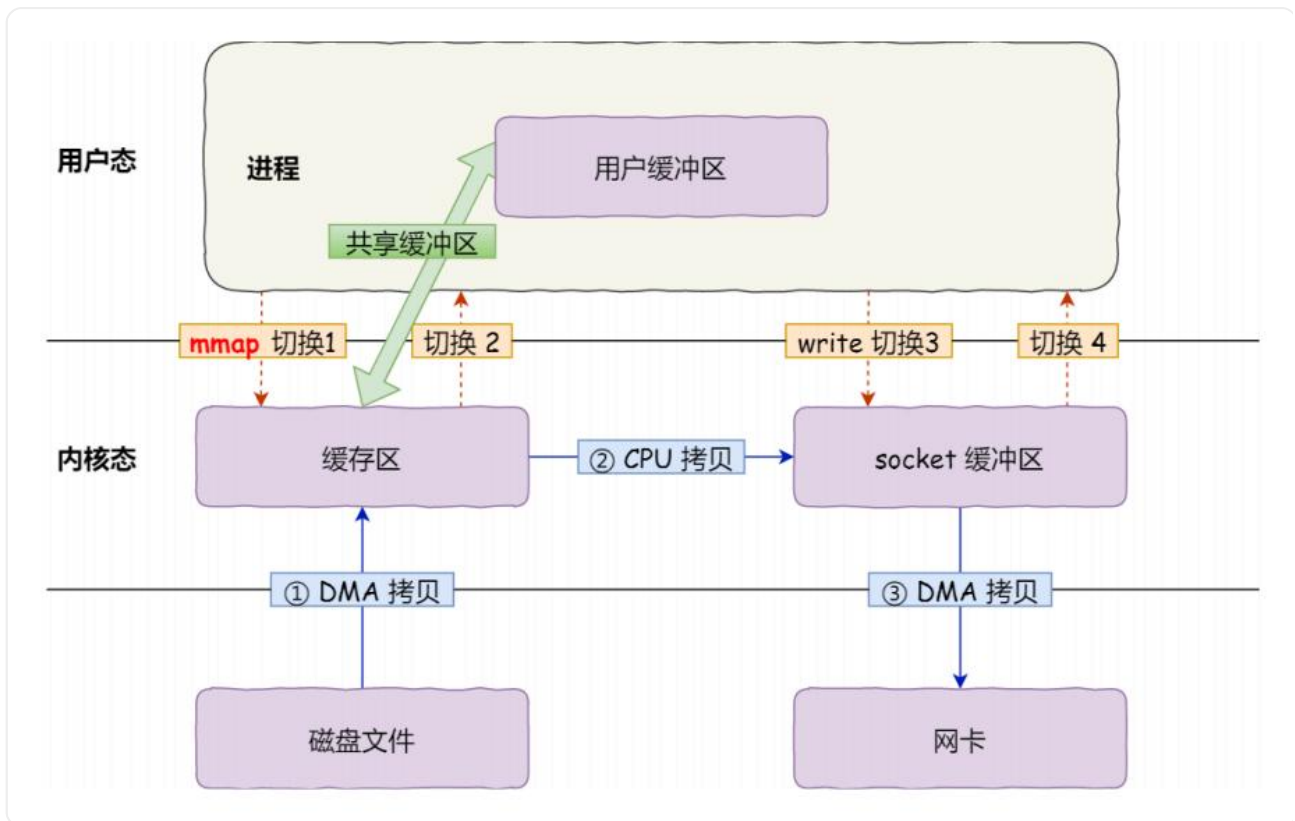
传统文件传输示意图-来源参考[3]

为了提升 I/O 性能，就需要**减少用户态与内核态的上下文切换和内存拷贝的次数**。

这就用到了我们零拷贝的技术，零拷贝技术实现主要有两种：

- **mmap + write**

mmap() 系统调用函数会直接把内核缓冲区里的数据「**映射**」到用户空间，这样，操作系统内核与用户空间就不需要再进行任何的数据拷贝操作。



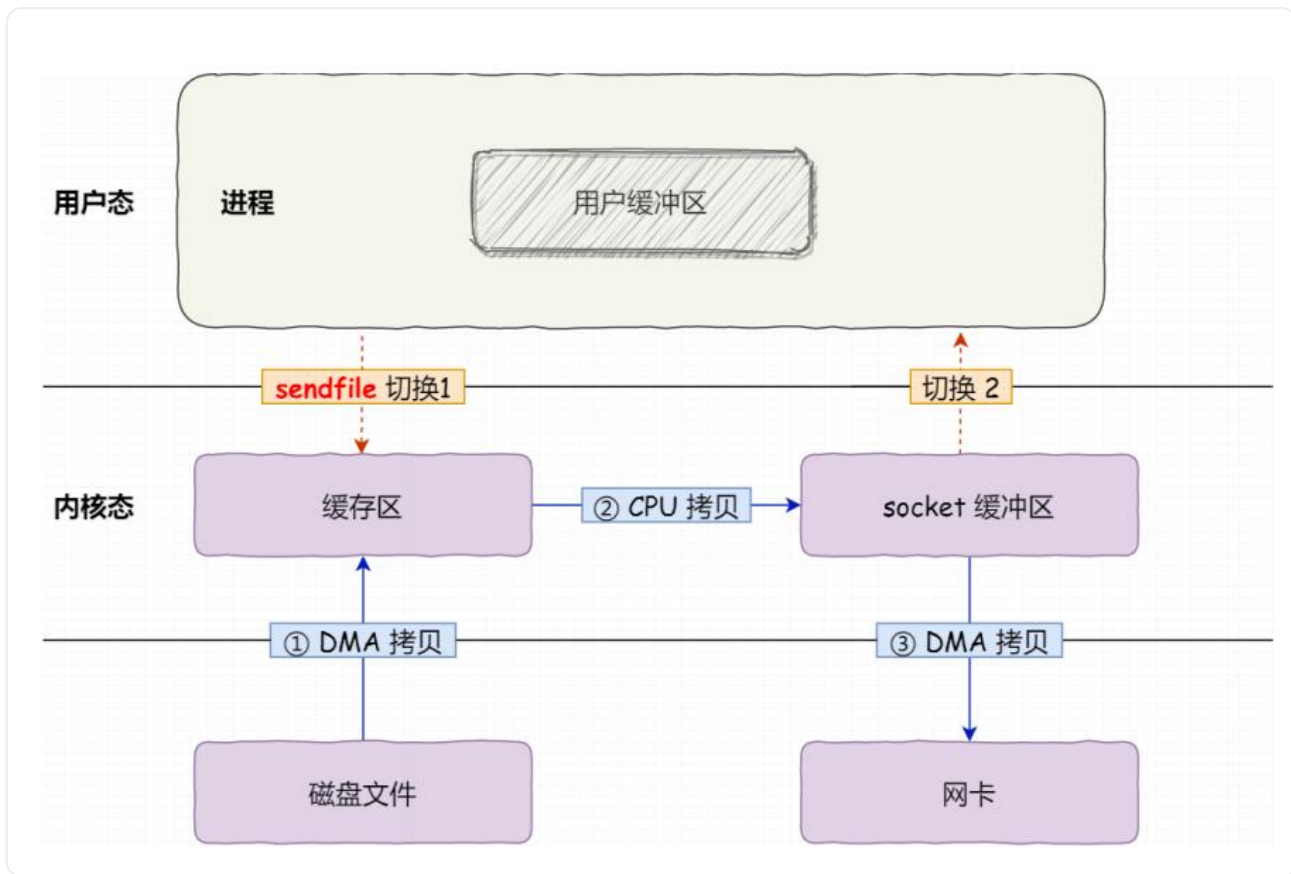
mmap示意图-来源参考[3]

• sendfile

在 Linux 内核版本 2.1 中，提供了一个专门发送文件的系统调用函数 `sendfile()`。

首先，它可以替代前面的 `read()` 和 `write()` 这两个系统调用，这样就可以减少一次系统调用，也就减少了 2 次上下文切换的开销。

其次，该系统调用，可以直接把内核缓冲区里的数据拷贝到 `socket` 缓冲区里，不再拷贝到用户态，这样就只有 2 次上下文切换，和 3 次数据拷贝。



sendfile示意图-来源参考[3]

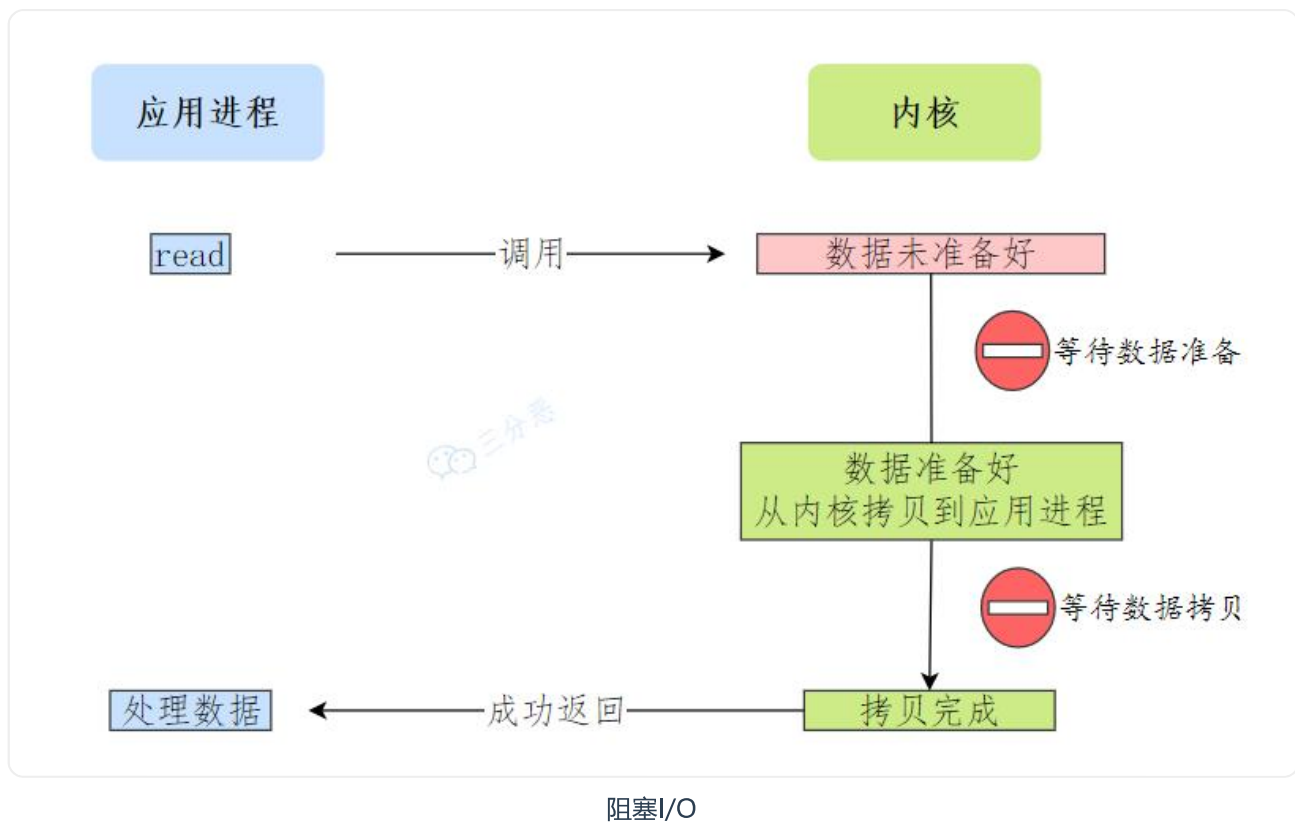
很多开源项目如 Kafka、RocketMQ 都采用了零拷贝技术来提升 IO 效率。

31、聊聊阻塞与非阻塞 IO、同步与异步 IO？

• 阻塞 I/O

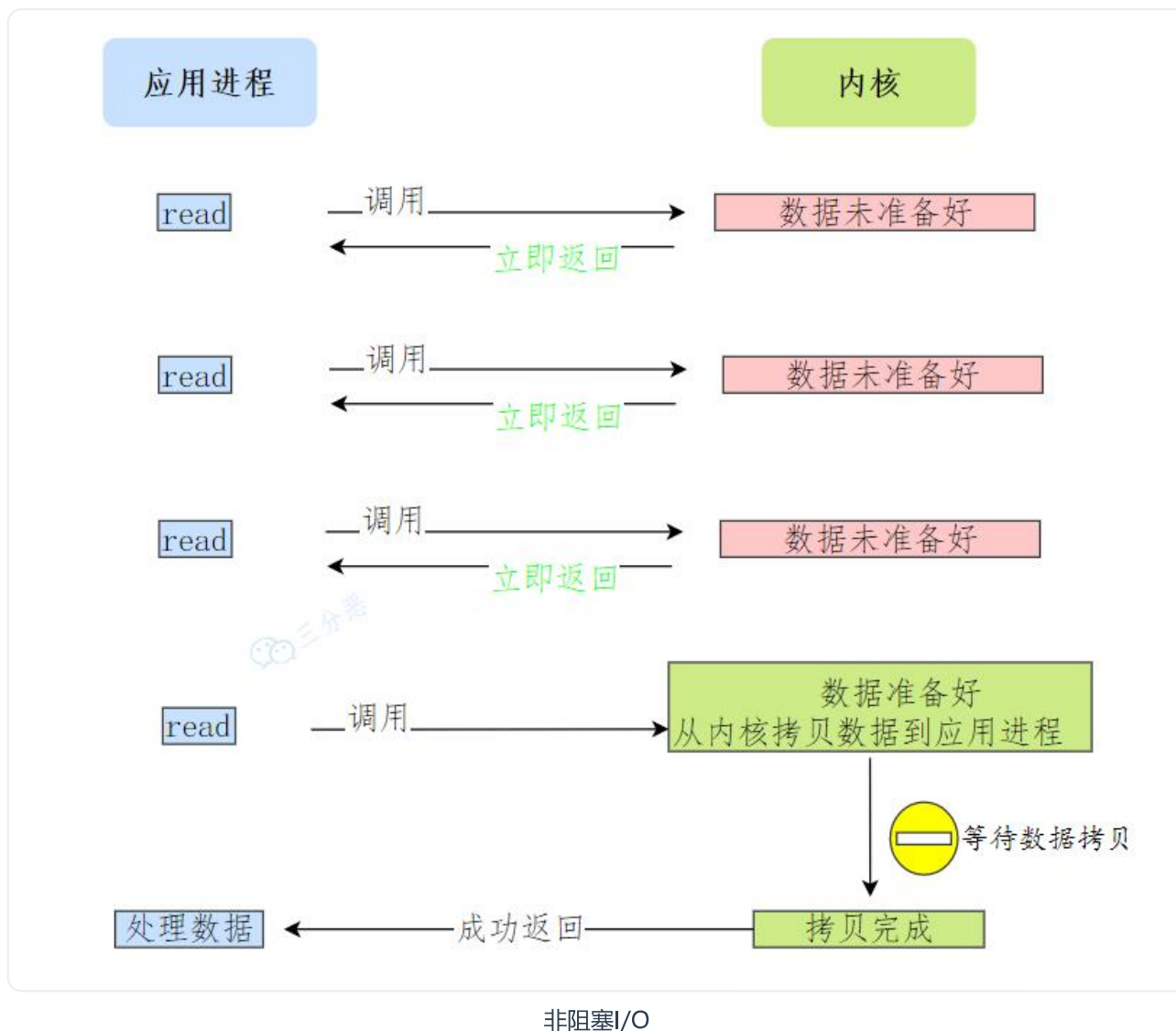
先来看看**阻塞 I/O**，当用户程序执行 `read`，线程会被阻塞，一直等到内核数据准备好，并把数据从内核缓冲区拷贝到应用程序的缓冲区中，当拷贝过程完成，`read` 才会返回。

注意，**阻塞等待的是** 内核数据准备好 **和** 数据从内核态拷贝到用户态 **这两个过程**。



• 非阻塞 I/O

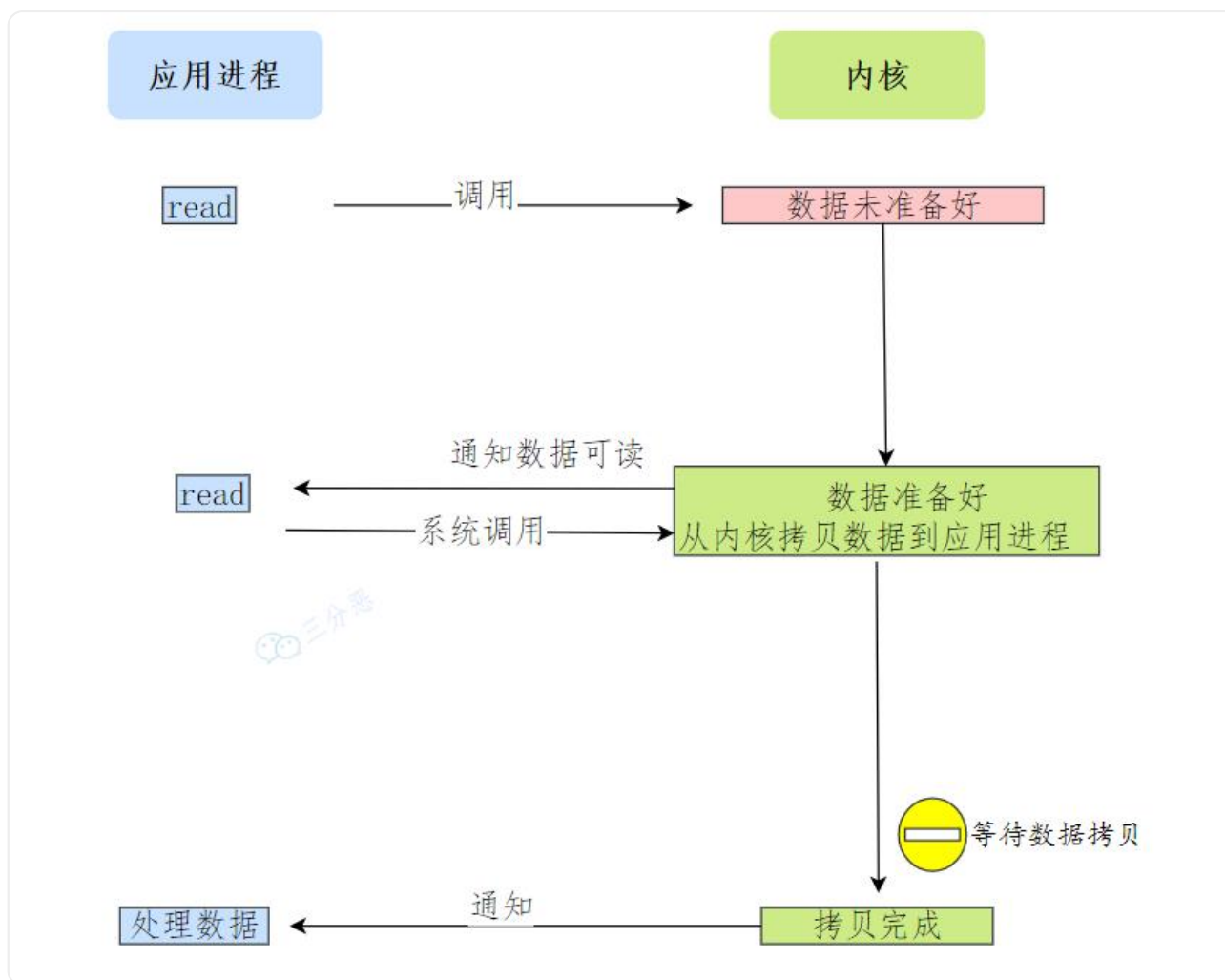
非阻塞的 `read` 请求在数据未准备好的情况下立即返回，可以继续往下执行，此时应用程序不断轮询内核，直到数据准备好，内核将数据拷贝到应用程序缓冲区，`read` 调用才可以获取到结果。



• 基于非阻塞的 I/O 多路复用

我们上面的非阻塞 I/O 有一个问题，什么问题呢？应用程序要一直轮询，这个过程没法干其它事情，所以引入了**I/O 多路复用**技术。

当内核数据准备好时，以事件通知应用程序进行操作。



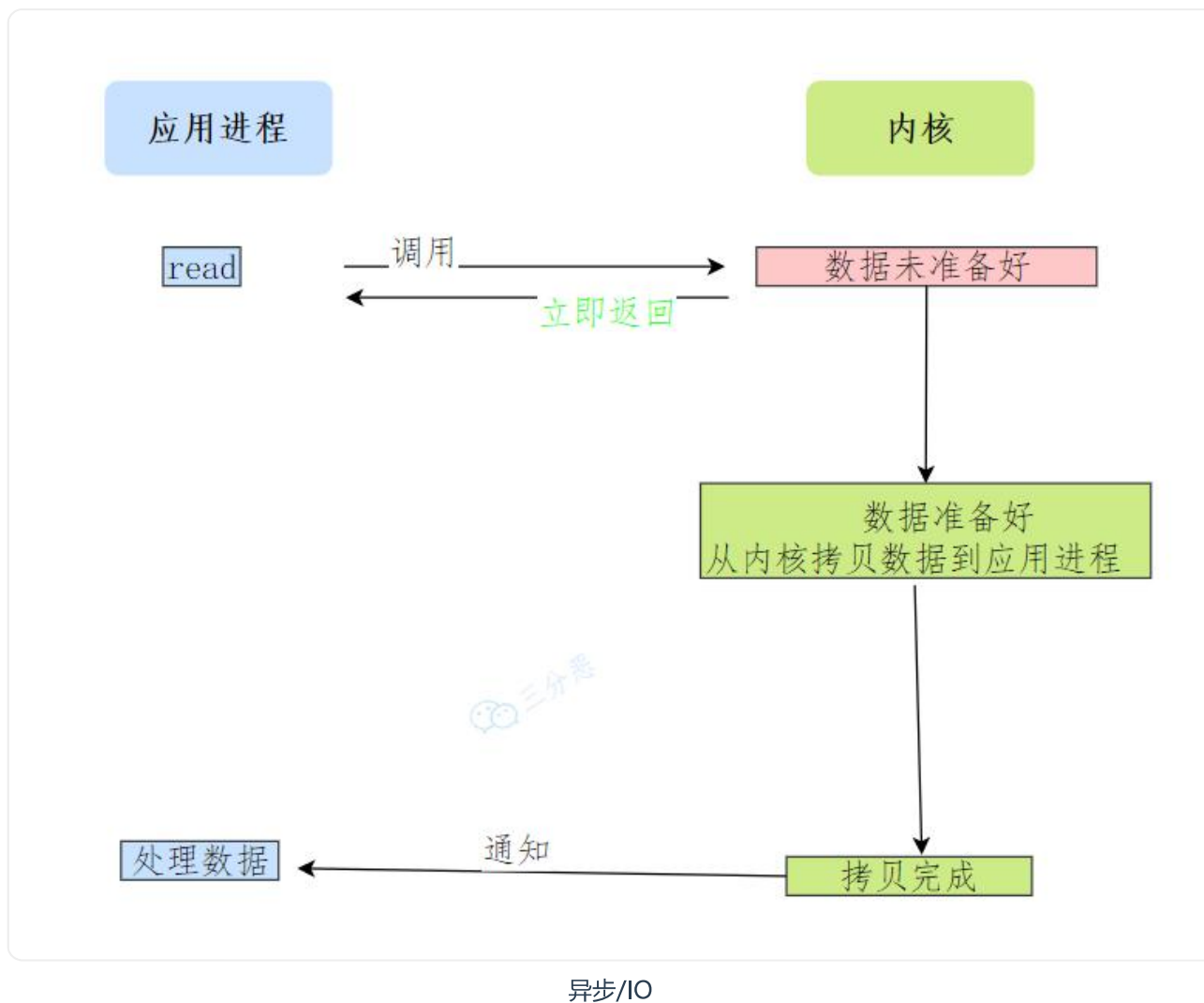
基于非阻塞的I/O多路复用

注意：无论是阻塞 I/O、还是非阻塞 I/O、非阻塞 I/O 多路复用，都是同步调用。因为它们 在 `read` 调用时，内核将数据从内核空间拷贝到应用程序空间，过程都是需要等待的，也就是说这个过程是同步的，如果内核实现的拷贝效率不高，`read` 调用就会在这个同步过程中等待比较长的时间。

• 异步 I/O

真正的**异步 I/O**是 内核数据准备好 和 数据从内核态拷贝到用户态 这两个过程都不用等待。

发起 `aio_read` 之后，就立即返回，内核自动将数据从内核空间拷贝到应用程序空间，这个拷贝过程同样是异步的，内核自动完成的，和前面的同步操作不一样，应用程序并不需要主动发起拷贝动作。



拿例子理解几种 I/O 模型

老三关注了很多 UP 主，有些 UP 主是老鸽子，到了更新的时间：

阻塞 I/O 就是，老三不干别的，就干等着，盯着 UP 的更新。

非阻塞 I/O 就是，老三发现 UP 没更，就去喝个茶什么的，过一会儿来盯一次，一直等到 UP 更新。

基于非阻塞的 I/O 多路复用好比，老三发现 UP 没更，就去干别的，过了一会儿 B 站推送消息了，老三一看，有很多条，就去翻动态，看看等的 UP 是不是更新了。

异步 I/O 就是，老三说 UP 你该更了，UP 赶紧爆肝把视频做出来，然后把视频亲自呈到老三面前，这个过程不用等待。



32、详细讲一讲 I/O 多路复用？

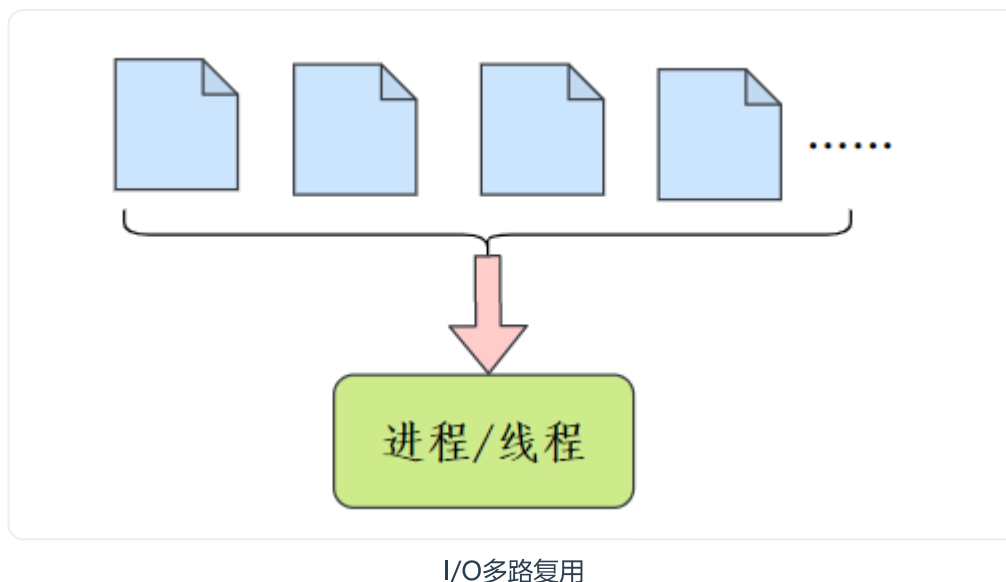
我们先了解什么是 I/O 多路复用？

我们在传统的 I/O 模型中，如果服务端需要支持多个客户端，我们可能要为每个客户端分配一个进程/线程。

不管是基于重一点的进程模型，还是轻一点的线程模型，假如连接多了，操作系统是扛不住的。

所以就引入了**I/O 多路复用** 技术。

简单说，就是一个进程/线程维护多个 Socket，这个多路复用就是多个连接复用一个进程/线程。



我们来看看 I/O 多路复用三种实现机制：

- **select**

select 实现多路复用的方式是：

将已连接的 Socket 都放到一个**文件描述符集合**fd_set，然后调用 select 函数将 fd_set 集合拷贝到内核里，让内核来检查是否有网络事件产生，检查的方式很粗暴，就是通过遍历 fd_set 的方式，当检查到有事件产生后，将此 Socket 标记为可读或可写，接着再把整个 fd_set 拷贝回用户态里，然后用户态还需要再通过遍历的方法找到可读或可写的 Socket，再对其处理。

select 使用固定长度的 Bitmap，表示文件描述符集合，而且所支持的文件描述符的个数是有限制的，在 Linux 系统中，由内核中的 FD_SETSIZE 限制，默认最大值为 1024，只能监听 0~1023 的文件描述符。

select 机制的缺点：

(1) 每次调用 select，都需要把 fd_set 集合从用户态拷贝到内核态，如果 fd_set 集合很大时，那这个开销也很大，比如百万连接却只有少数活跃连接时这样做就太没有效率。

(2) 每次调用 select 都需要在内核遍历传递进来的所有 fd_set，如果 fd_set 集合很大时，那这个开销也很大。

(3) 为了减少数据拷贝带来的性能损坏，内核对被监控的 fd_set 集合大小做了限制，一般为 1024，如果想要修改会比较麻烦，可能还需要编译内核。

(4) 每次调用 select 之前都需要遍历设置监听集合，重复工作。

- **poll**

`poll` 不再用 `Bitmap` 来存储所关注的文件描述符，取而代之用动态数组，以链表形式来组织，突破了 `select` 的文件描述符个数限制，当然还会受到系统文件描述符限制。

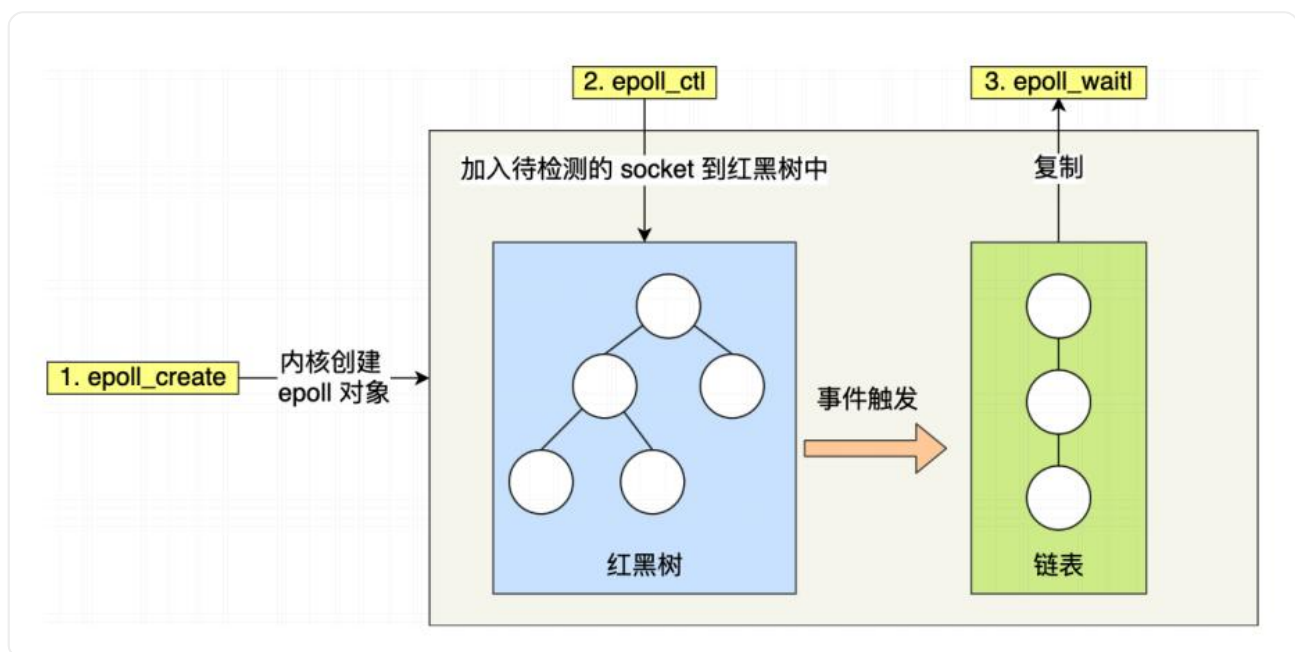
但是 `poll` 和 `select` 并没有太大的本质区别，都是使用线性结构存储进程关注的 `Socket` 集合，因此都需要遍历文件描述符集合来找到可读或可写的 `Socket`，时间复杂度为 $O(n)$ ，而且也需要在用户态与内核态之间拷贝文件描述符集合，这种方式随着并发数上来，性能的损耗会呈指数级增长。

• `epoll`

`epoll` 通过两个方面，很好解决了 `select/poll` 的问题。

第一点，`epoll` 在内核里使用**红黑树来跟踪进程所有待检测的文件描述符**，把需要监控的 `socket` 通过 `epoll_ctl()` 函数加入内核中的红黑树里，红黑树是个高效的数据结构，增删查一般时间复杂度是 $O(\log n)$ ，通过对这棵黑红树进行操作，这样就不需要像 `select/poll` 每次操作时都传入整个 `socket` 集合，只需要传入一个待检测的 `socket`，**减少了内核和用户空间大量的数据拷贝和内存分配**。

第二点，`epoll` 使用事件驱动的机制，内核里**维护了一个链表来记录就绪事件**，当某个 `socket` 有事件发生时，通过回调函数，内核会将其加入到这个就绪事件列表中，当用户调用 `epoll_wait()` 函数时，只会返回有事件发生的文件描述符的个数，不需要像 `select/poll` 那样轮询扫描整个 `socket` 集合，大大提高了检测的效率。



`epoll`接口作用-来源参考[3]

`epoll` 的方式即使监听的 `Socket` 数量越来越多的时候，效率不会大幅度降低，能够同时监听的 `Socket` 的数目也非常的多了，上限就为系统定义的进程打开的最大文件描述符个数。因而，**`epoll` 被称为解决 C10K 问题的利器**。

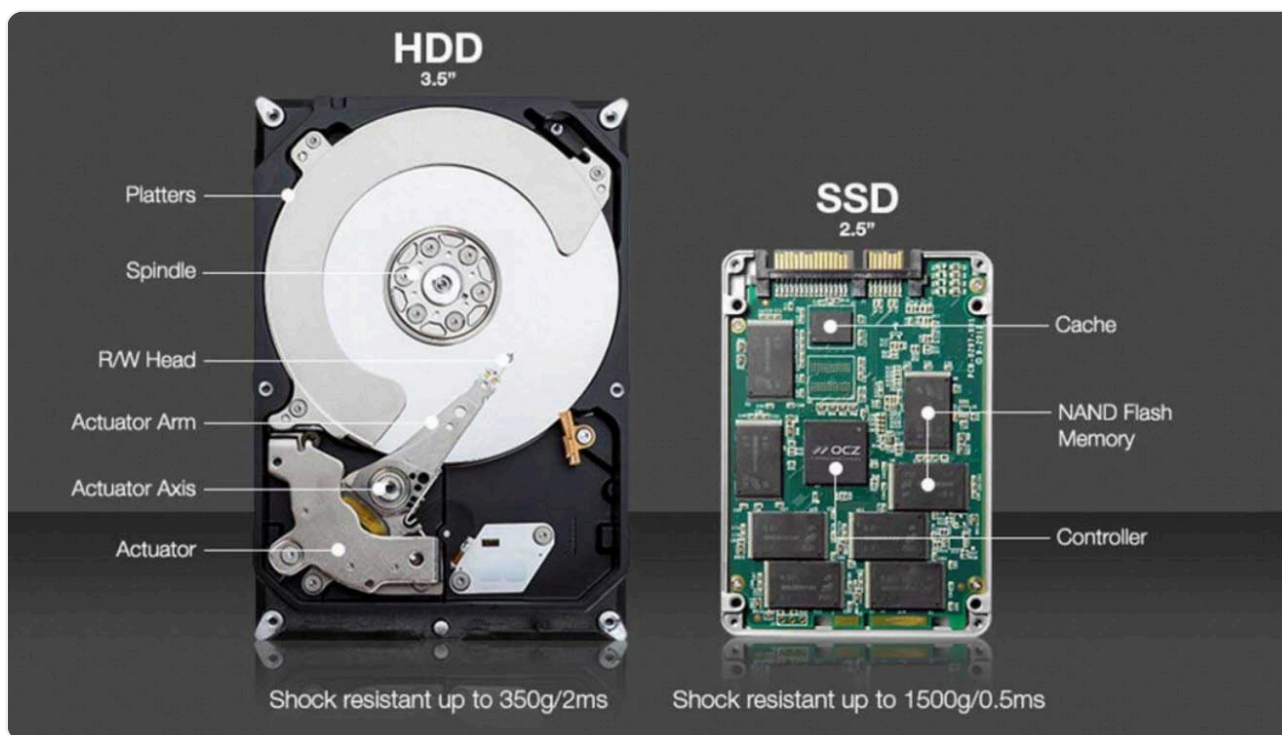
34.普通内存比一般的机械硬盘快多少？（补充）

2024 年 04 月 10 日增补

机械硬盘，也叫 HDD (Hard Disk Drive)，是一种通过磁盘旋转和磁头移动来存储数据的设备，读写速度比较慢，通常比内存的速度慢 10 万左右。

- HDD 的访问时间大约在 5-10ms，数据传输速率约为 100 到 200 MB/s。
- 内存，也就是 RAM (Random Access Memory)，访问时间大约在 10-100ns，数据传输速率约为数十 GB/s。

固态硬盘 (Solid State Drive, SSD)，SSD 的读写速度比 HDD 快 200 倍左右，价格也在逐渐下降，已经逐渐取代了 HDD。



图片来源于网络

1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的奇安信面经同学 1 Java 技术一面面试原题：普通内存比一般的机械硬盘快多少？

图文详解 34 道操作系统面试高频题，这次吊打面试官，我觉得稳了（手动 dog）。整理：沉默王二，戳[转载链接](https://mp.weixin.qq.com/s/CYsn0M5ddDuG--mAlmhsw) (<https://mp.weixin.qq.com/s/CYsn0M5ddDuG--mAlmhsw>)，作者：三分恶，戳[原文链接](https://mp.weixin.qq.com/s/KMGyn-) (<https://mp.weixin.qq.com/s/KMGyn->

[FLkvzsMH06LV4OfQ\)_](#)。

没有什么使我停留——除了目的，纵然岸旁有玫瑰、有绿荫、有宁静的港湾，我是不系之舟。

系列内容：

- [面渣逆袭 Java SE 篇👉](https://javabetter.cn/sidebar/sanfene/javase.html) (<https://javabetter.cn/sidebar/sanfene/javase.html>)
- [面渣逆袭 Java 集合框架篇👉](https://javabetter.cn/sidebar/sanfene/javathread.html) (<https://javabetter.cn/sidebar/sanfene/javathread.html>)
- [面渣逆袭 Java 并发编程篇👉](https://javabetter.cn/sidebar/sanfene/collection.html) (<https://javabetter.cn/sidebar/sanfene/collection.html>)
- [面渣逆袭 JVM 篇👉](https://javabetter.cn/sidebar/sanfene/jvm.html) (<https://javabetter.cn/sidebar/sanfene/jvm.html>)
- [面渣逆袭 Spring 篇👉](https://javabetter.cn/sidebar/sanfene/spring.html) (<https://javabetter.cn/sidebar/sanfene/spring.html>)
- [面渣逆袭 Redis 篇👉](https://javabetter.cn/sidebar/sanfene/redis.html) (<https://javabetter.cn/sidebar/sanfene/redis.html>)
- [面渣逆袭 MyBatis 篇👉](https://javabetter.cn/sidebar/sanfene/mybatis.html) (<https://javabetter.cn/sidebar/sanfene/mybatis.html>)
- [面渣逆袭 MySQL 篇👉](https://javabetter.cn/sidebar/sanfene/mysql.html) (<https://javabetter.cn/sidebar/sanfene/mysql.html>)
- [面渣逆袭操作系统篇👉](https://javabetter.cn/sidebar/sanfene/os.html) (<https://javabetter.cn/sidebar/sanfene/os.html>)
- [面渣逆袭计算机网络篇👉](https://javabetter.cn/sidebar/sanfene/network.html) (<https://javabetter.cn/sidebar/sanfene/network.html>)
- [面渣逆袭 RocketMQ 篇👉](https://javabetter.cn/sidebar/sanfene/rocketmq.html) (<https://javabetter.cn/sidebar/sanfene/rocketmq.html>)
- [面渣逆袭分布式篇👉](https://javabetter.cn/sidebar/sanfene/fenbushi.html) (<https://javabetter.cn/sidebar/sanfene/fenbushi.html>)
- [面渣逆袭微服务篇👉](https://javabetter.cn/sidebar/sanfene/weifuwu.html) (<https://javabetter.cn/sidebar/sanfene/weifuwu.html>)
- [面渣逆袭设计模式篇👉](https://javabetter.cn/sidebar/sanfene/shejimoshi.html) (<https://javabetter.cn/sidebar/sanfene/shejimoshi.html>)
- [面渣逆袭 Linux 篇👉](https://javabetter.cn/sidebar/sanfene/linux.html) (<https://javabetter.cn/sidebar/sanfene/linux.html>)

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](#)

<https://github.com/itwanger/toBeBetterJavaer>》第一版 PDF 终于来了！包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等，共计 32 万余字，500+张手绘图，可以说是通俗易懂、风趣幽默.....详情戳：[太赞了，GitHub 上标星 10000+ 的 Java 教程](#) (<https://javabetter.cn/overview/>)

微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二，回复 **222** 即可免费领取。



上次编辑于: 2024/7/1 18:41:15

贡献者: itwanger,沉默王二,LJ

Copyright © 2024 三分恶